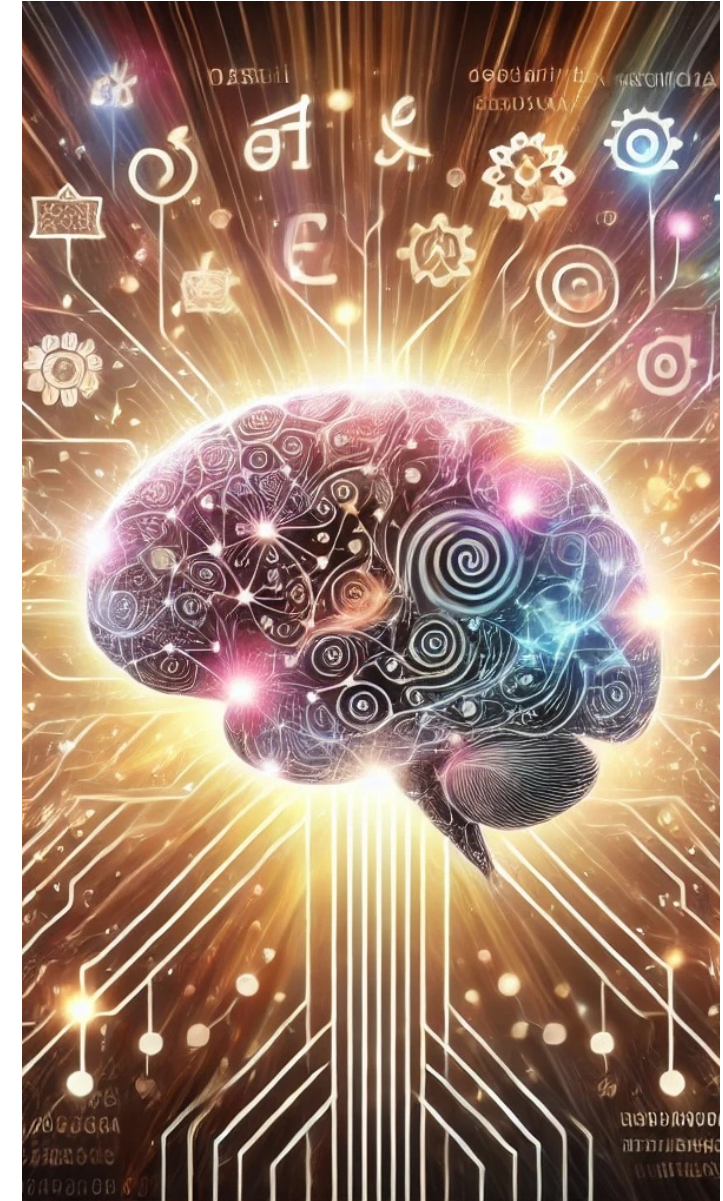


Chapter 09: Word Embeddings

Prachya Boonkwan (Arm)

School of ICT, SIIT,
Thammasat University

prachya@siit.tu.ac.th, kaamanita@gmail.com



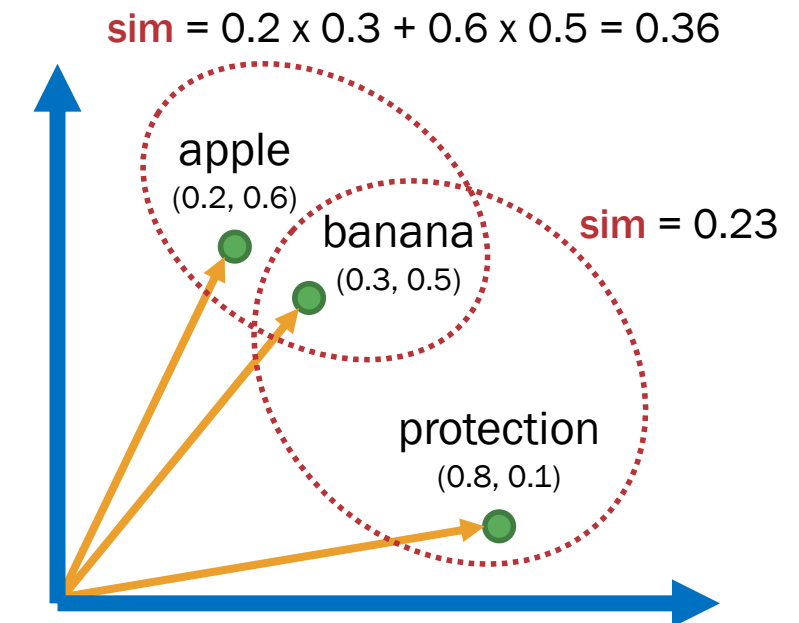
Who? Me?

- Nickname: **Arm** (P'/N' Arm, etc.)
- Born: Aug 1981
- Work
 - Researcher at NECTEC 2005-2024
 - Lecturer at SIIT, Thammasat University 2025-now
- Education
 - B.Eng & M.Eng, CPE Kasetsart University
 - Obtained Ministry of Science Scholarship in early 2008
 - Did a PhD in Informatics (AI & Computational Linguistics) at University of Edinburgh, UK from 2008 to 2013 (4.5 years)



In a Nutshell

- **Word embedding** (Mikolov+, 2013)
 - Vector representation of words w.r.t. their contexts (i.e. word = **point**) (Rumelhart+, 1986)
 - The dot product of two **word vectors** yields their spatial distance (i.e. **word similarity**)
 - In fact, this distance is pointwise mutual information (PMI) of the words (Levy+, 2014)
 - Word2Vec is Google's NLP tool for efficiently computing such vector representation



Question: What do these numeric elements of the word vectors represent?

Code: <http://code.google.com/p/word2vec>

Outline

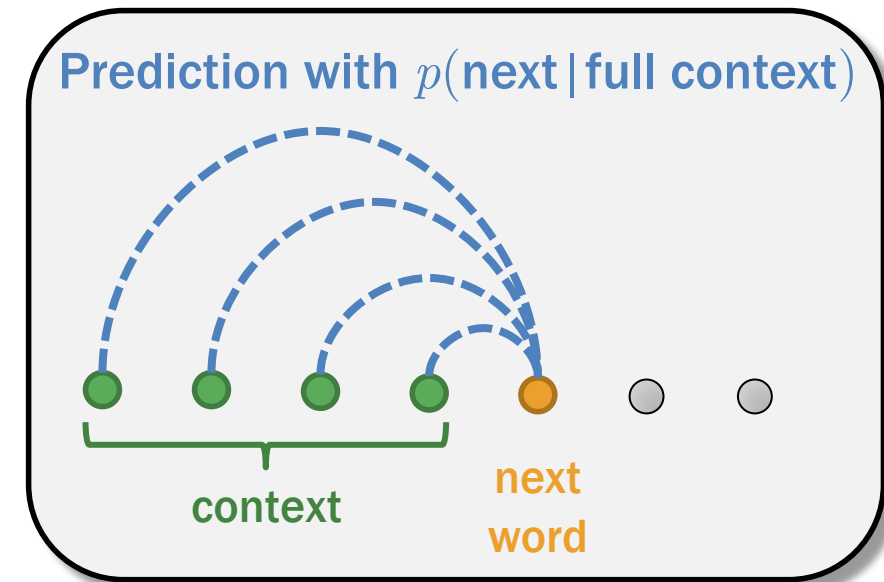
- Introduction
- Word embedding models
- Training with backpropagation
- Performance optimization
- Statistical interpretation
- Experiments as in literature

1. Introduction

1. Introduction

- **Motivation:** language modeling
 - Statistical prediction for how strings are produced in a language
 - Interpreted as a generative model
 1. Generate the first word w_1
 2. Keep generating the **next word** w_k based on the previous words (a.k.a. **context**) $w_1 \dots w_{k-1}$ until the whole sentence of length N is produced

$$P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^N p(\overbrace{w_k}^{\text{next word}} | \overbrace{w_1 \dots w_{k-1}}^{\text{context}})$$



1. Introduction (cont'd)

- Motivation:** n -gram models

N-1 words, predict the next one

*take all words and predict next
"A lot" "too much computation"*

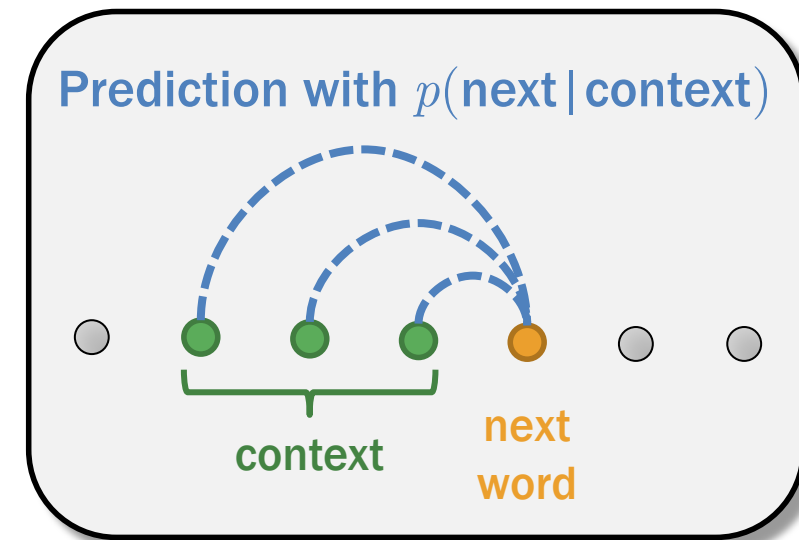
- Language models whose context is truncated to at most $n-1$ previous words

$$P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^N p(\overbrace{w_k}^{\text{next word}} | \overbrace{w_{k-n+1} \dots w_{k-1}}^{n-1 \text{ prev words}})$$

- Unigram** ($n=1$): $P(w_1 \dots w_N) = \prod_{k=1}^N p(w_k)$

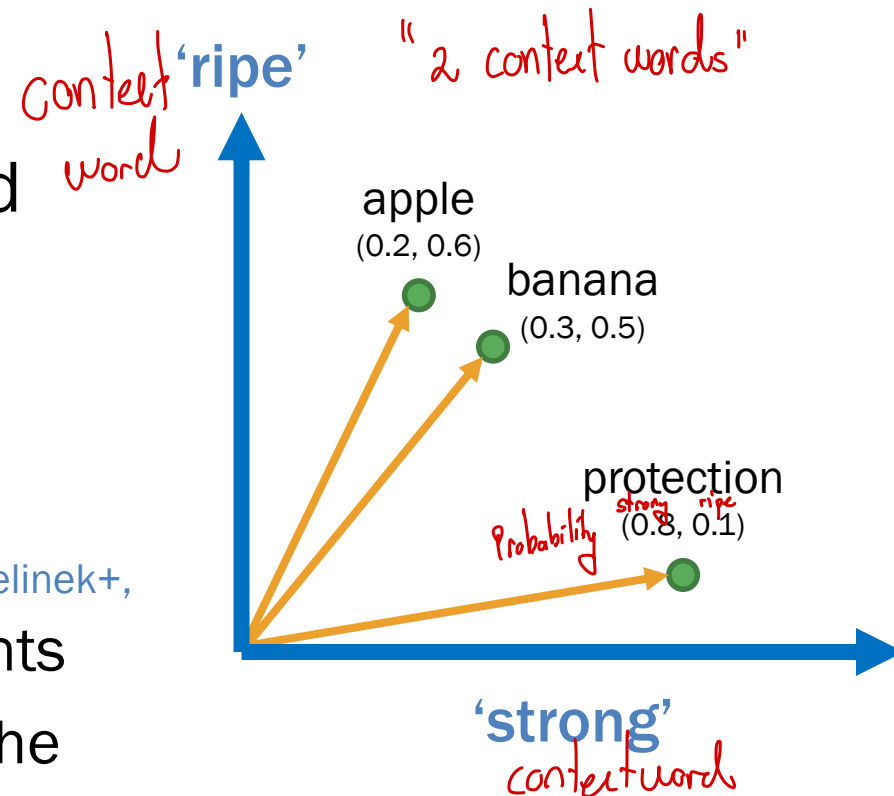
- Bigram** ($n=2$): $P(w_1 \dots w_N) = p(w_1) \prod_{k=2}^N p(w_k | w_{k-1})$

- Trigram** ($n=3$): $P(w_1 \dots w_N) = p(w_1)p(w_2|w_1) \prod_{k=3}^N p(w_k | w_{k-2}, w_{k-1})$



1. Introduction (cont'd)

- **Distributional similarity of words**
 - Measured by co-occurrence of words and their contexts (i.e. **context distribution**)
 - Various types of similarity metrics and context are employed (Dagan+, 2008)
 - Backoff method (Katz, 1987) with interpolation (Jelinek+, 1980) are required for smoothing the zero counts
 - Bigram co-occurrence is commonly used as the context representation (Brown+, 1992)
 - Well-known techniques include Latent Semantic Analysis (Dumais, 2005) and PMI (Church+, 1990)



In this over-simplified example, each numeric element is:

$$p(\text{word} | \text{context})$$

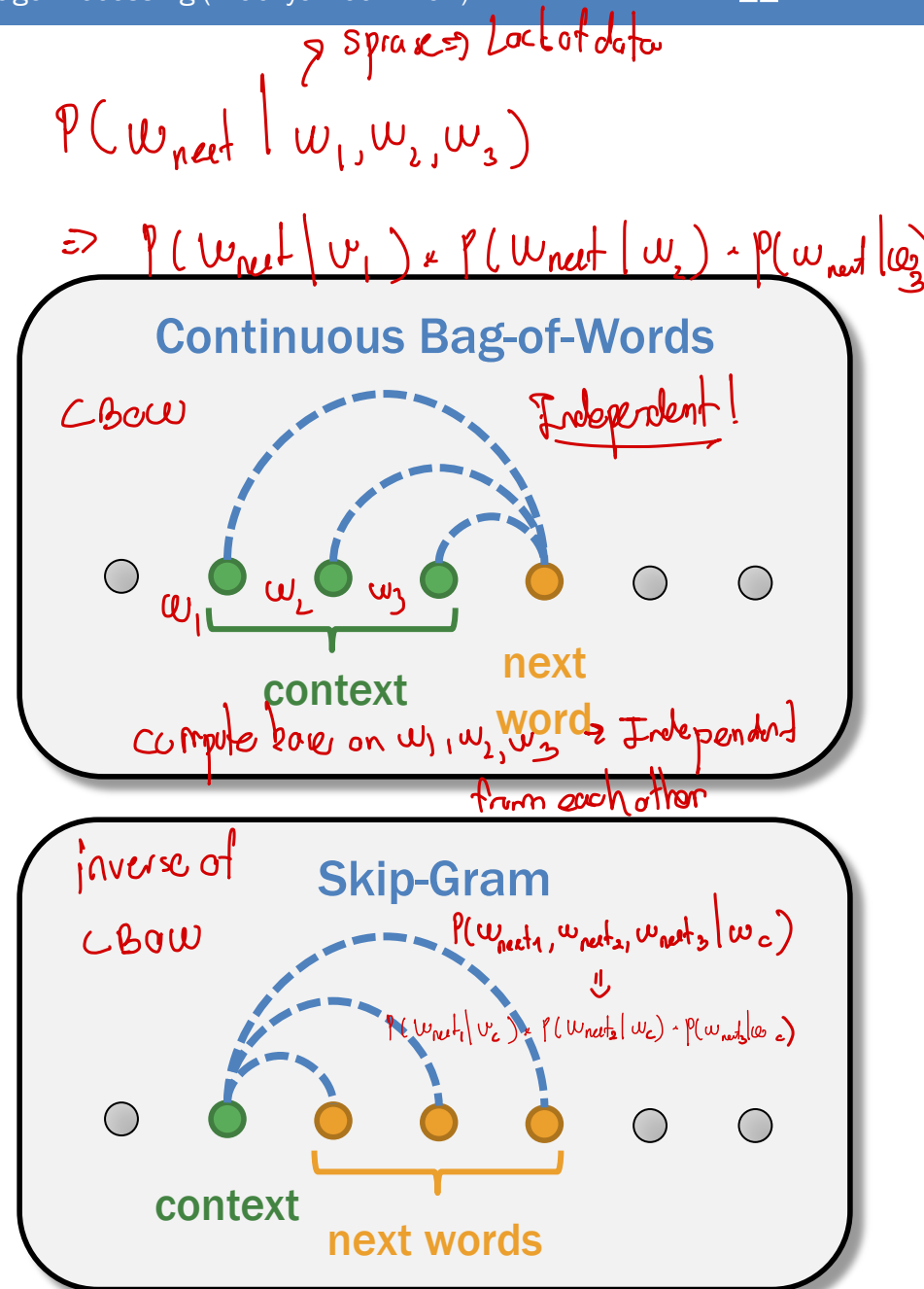
Problems

- **Coverage:** Co-occurrence should not be limited to consecutive words
- **Space:** Natural language vocabulary (700k+) normally lead to a large vector space $O(n^2)$ and sparse distribution (i.e. the curse of dimensionality)
- **Speed:** Maximum likelihood is very expensive when estimated on big data because of a large vector space

2. Word Embedding Models

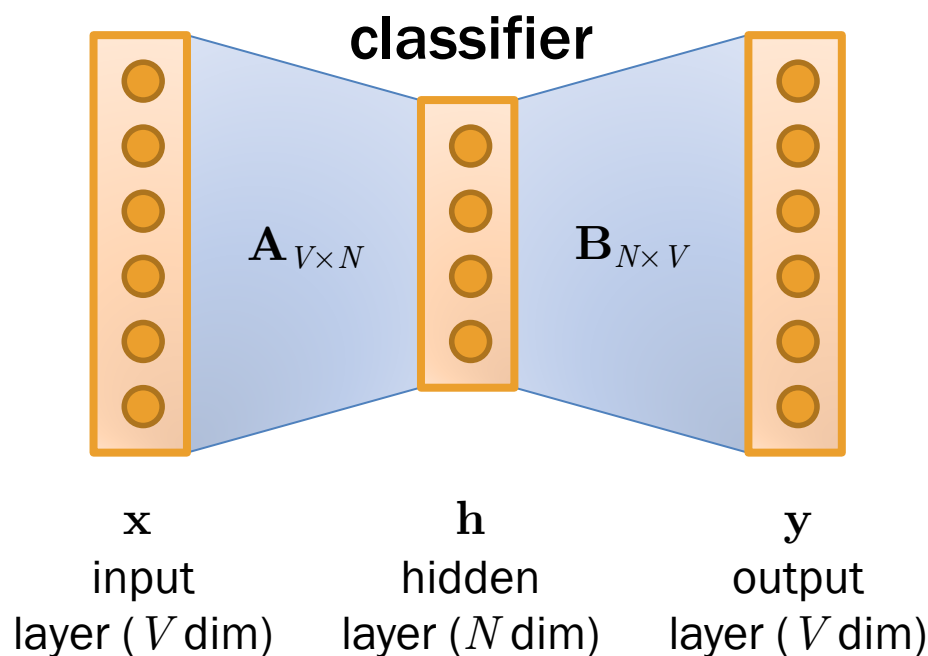
2. Word Embedding Models

- [P: Coverage] Co-occurrence should not be limited to consecutive words
- Location-less context representations
- **Continuous Bag-of-Words** (Harris, 1954): Many context words generate the next word
- **Skip-Gram** (Mikolov+, 2013): One context word generates many next words
- Both can be efficiently estimated with artificial neural networks (Bengio+, 2003)



2. Word Embedding Models (cont'd)

- Neural network for word co-occurrence

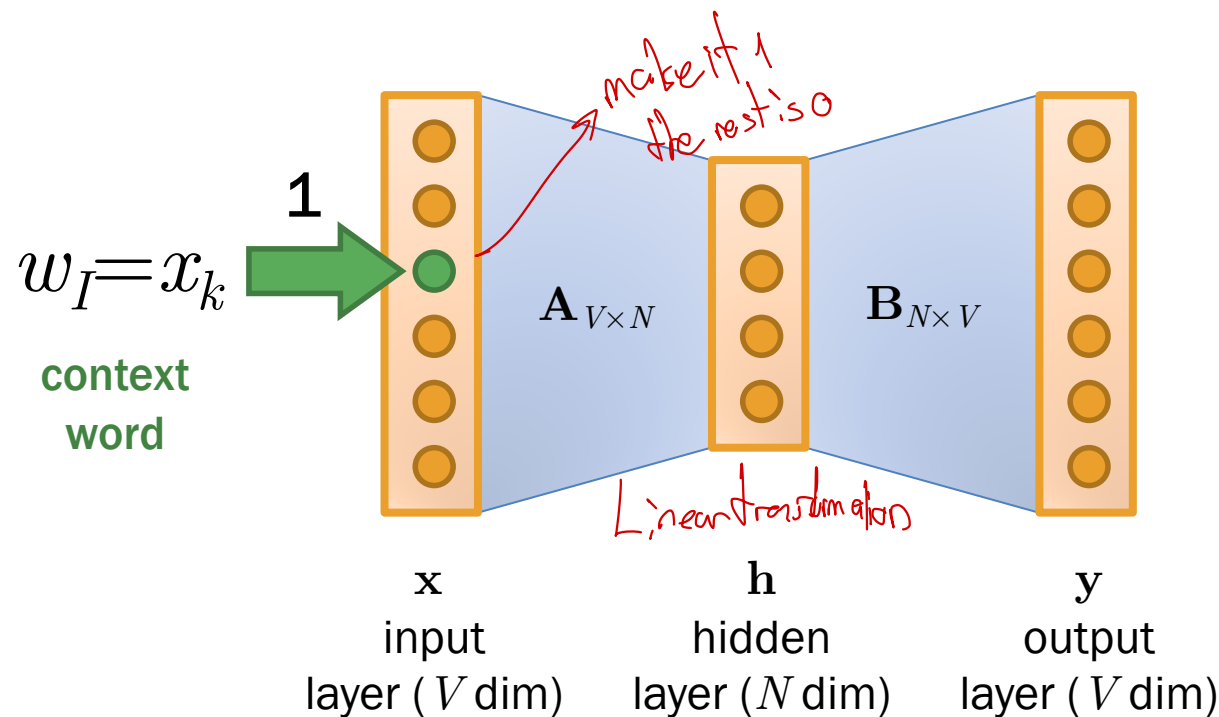


One-hot vector
 $[0, 0, 0, 1]$
 $[1, 0, 0, 0]$

- The hidden layer acts as a classifier, because it reduces the input layer's dimensionality to N
- The next word is predicted by the softmax distribution over the output layer

2. Word Embedding Models (cont'd)

- How it works: feedforwarding

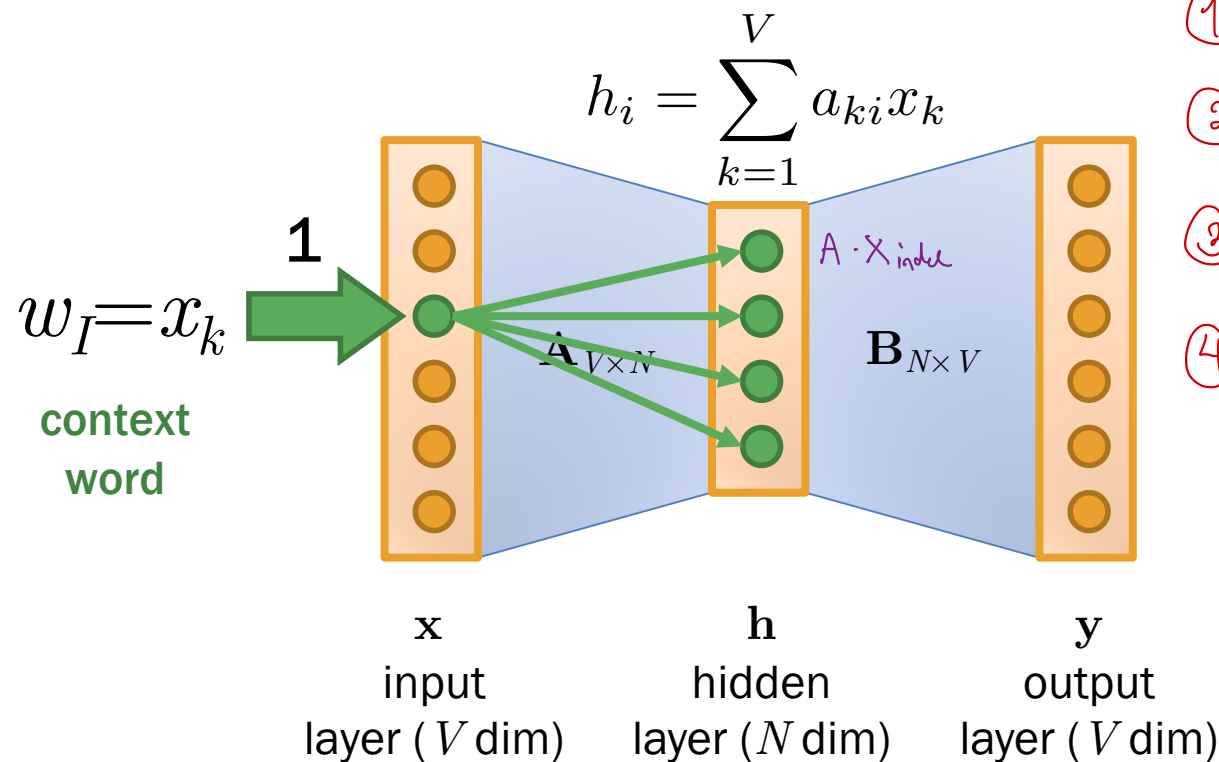


- The hidden layer acts as a classifier, because it reduces the input layer's dimensionality to N
- The next word is predicted by the softmax distribution over the output layer

(1) Let the input be a one-hot layer

2. Word Embedding Models (cont'd)

- How it works: feedforwarding



① $\bar{x}$ one hot vector

② $\bar{h} = \bar{A} \bar{x}$ hidden vector

③ $\bar{w} = \bar{B} \bar{h}$ output vector

④ next word $j^* = \text{argmax}_j w_j$

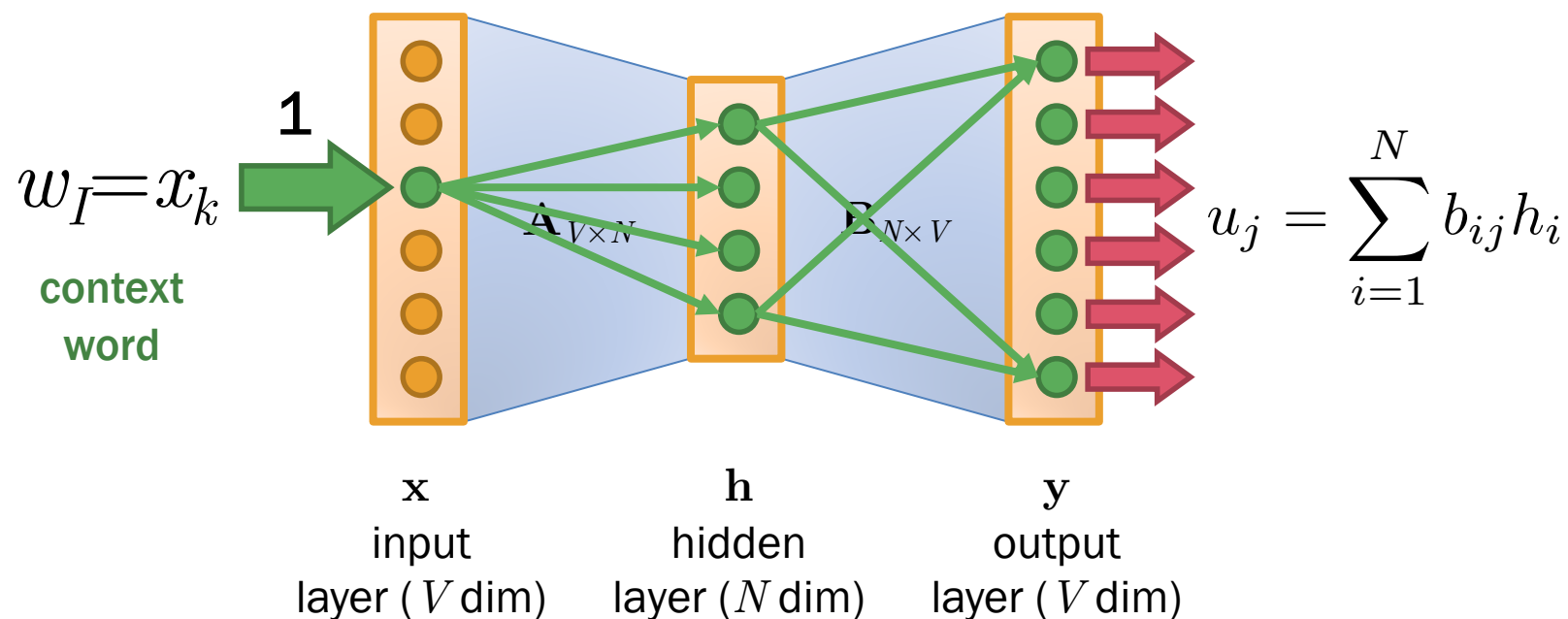
- The hidden layer acts as a classifier, because it reduces the input layer's dimensionality to N

The next word is predicted by the softmax distribution over the output layer

(2) Compute each hidden unit h_i

2. Word Embedding Models (cont'd)

- How it works: feedforwarding

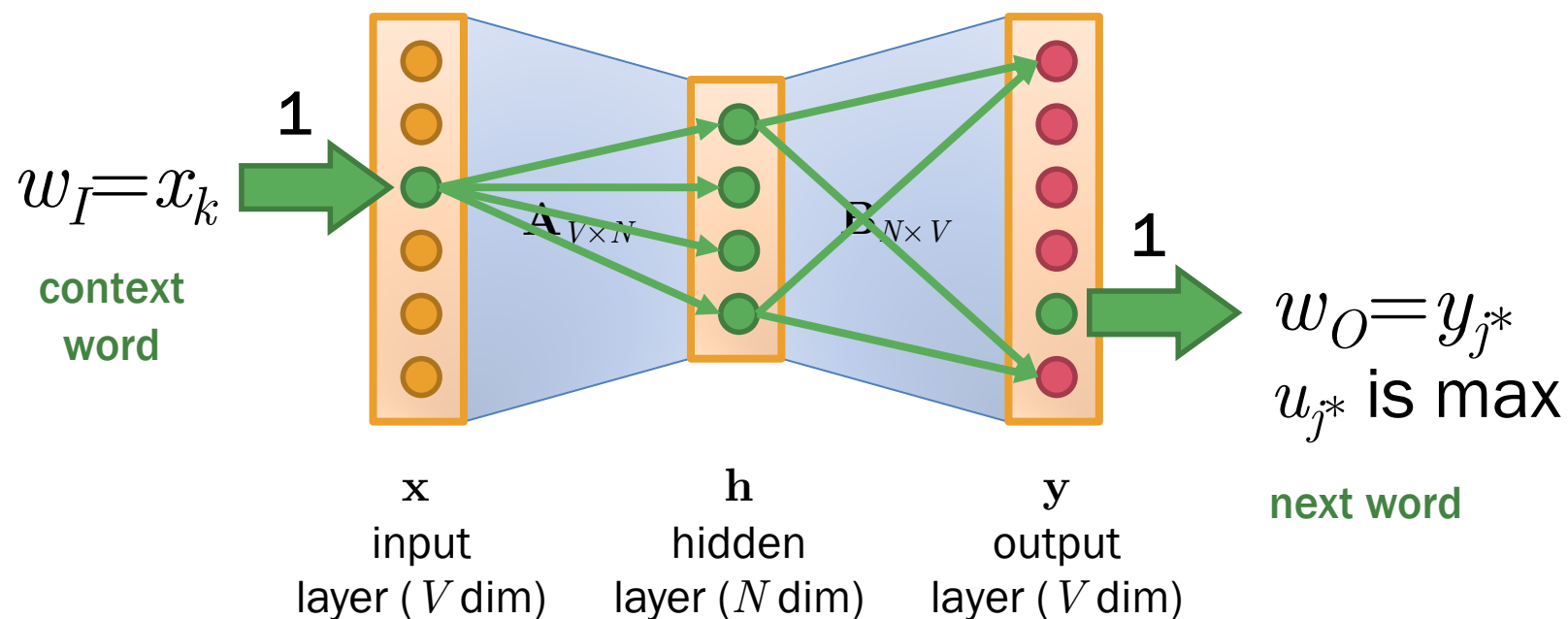


- The hidden layer acts as a classifier, because it reduces the input layer's dimensionality to N
- The next word is predicted by the softmax distribution over the output layer

(3) Compute each prediction score u_j

2. Word Embedding Models (cont'd)

- How it works: feedforwarding



- The hidden layer acts as a classifier, because it reduces the input layer's dimensionality to N
- The next word is predicted by the softmax distribution over the output layer

(4) Choose the output y_{j^*} based on softmax

2. Word Embedding Models (cont'd)

- Neural network for word co-occurrence

- Hidden unit:

$$h_i = \sum_{k=1}^V a_{ki}^T x_k$$

$\bar{x}$ (V dim) Dimensionality reduction

$N \times V$ dimension
 $\hookrightarrow$ Dim of your choice.

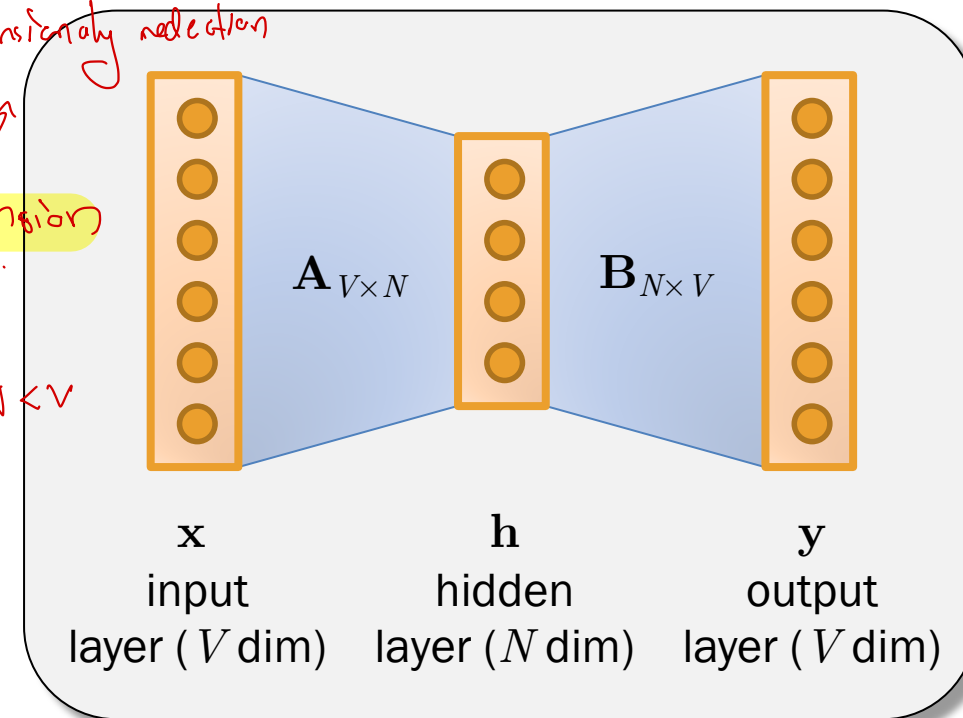
- Prediction score:

$$u_j = \sum_{i=1}^N b_{ij}^T h_i$$

$N < V$

- Posterior: $p(w_O = y_{j^*} | w_I) = \frac{\exp u_{j^*}}{\sum_{j=1}^V \exp u_j}$

(softmax distribution)
probability distributions.



$$\bar{u} = \text{softmax}(\bar{B}^T \bar{A}^T \bar{x})$$

2. Word Embedding Models (cont'd)

$$A = V \times N, A^T = N \times V$$

- Neural network for word co-occurrence

Matrix Form

- Hidden unit:

$$h_i = \sum_{k=1}^V a_{ki} x_k \quad \Rightarrow \quad \mathbf{h} = \mathbf{A}^T \mathbf{x}$$

- Prediction score:

$$u_j = \sum_{i=1}^N b_{ij} h_i \quad \Rightarrow \quad \mathbf{u} = \mathbf{B}^T \mathbf{h}$$

- Posterior: $p(w_O = y_{j^*} | w_I) = \frac{\exp u_{j^*}}{\sum_{j=1}^V \exp u_j}$
(softmax distribution)

2. Word Embedding Models (cont'd)

- We can reduce matrix multiplication into vector multiplication

Hidden Units (k^{th} one hot)

$$\begin{bmatrix} a_{1,1} & \dots & a_{k,1} & \dots & a_{V,1} \\ \vdots & & a_{k,2} & & \vdots \\ \vdots & & a_{k,3} & & \vdots \\ \vdots & & \vdots & & \vdots \\ a_{1,N} & \dots & a_{k,N} & \dots & a_{V,N} \end{bmatrix}_{N \times V} \times \begin{bmatrix} 0 \\ \vdots \\ 1 \\ \vdots \\ 0 \end{bmatrix}_{V \times 1}$$

$\mathbf{A}^\top$ $\mathbf{x}(k)$
 $\mathbf{v}_I(k)$

$$\mathbf{h}(k) = \mathbf{A}^\top \mathbf{x}(k) = \mathbf{v}_I(k)$$

Prediction Scores (fix on j)

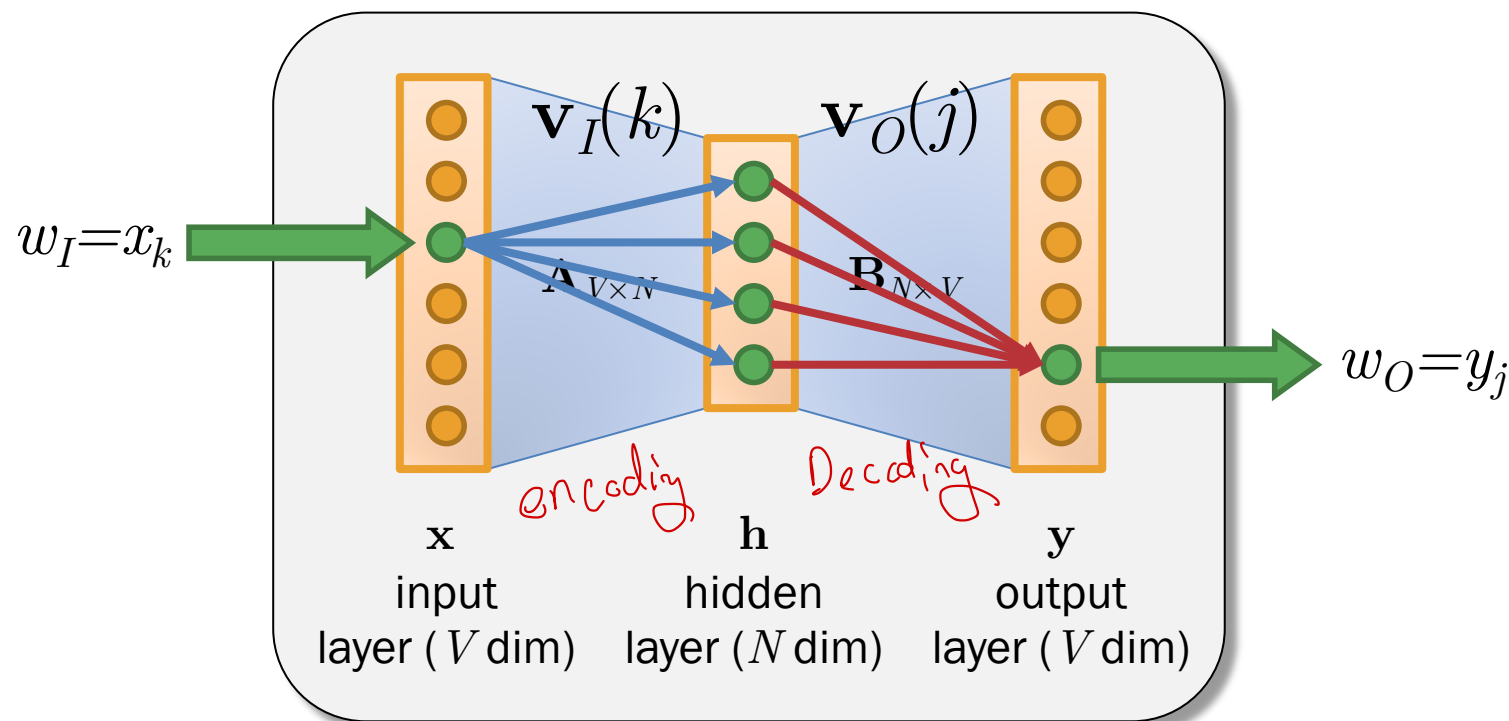
$$\begin{bmatrix} b_{1,1} & b_{1,2} & b_{1,3} & \dots & b_{1,N} \\ \vdots & & & & \vdots \\ b_{j,1} & b_{j,2} & b_{j,3} & \dots & b_{j,N} \\ \vdots & & & & \vdots \\ b_{V,1} & b_{V,2} & b_{V,3} & \dots & b_{V,N} \end{bmatrix}_{V \times N} \times \begin{bmatrix} h_1 \\ \vdots \\ \vdots \\ \vdots \\ h_N \end{bmatrix}_{N \times 1}$$

$\mathbf{B}^\top$ $\mathbf{h}(k)$
 $\mathbf{v}_O(j)^\top$

$$u_j = \mathbf{B}^\top \mathbf{h}(k) = \mathbf{v}_O(j)^\top \mathbf{v}_I(k)$$

dot product

2. Word Embedding Models (cont'd)



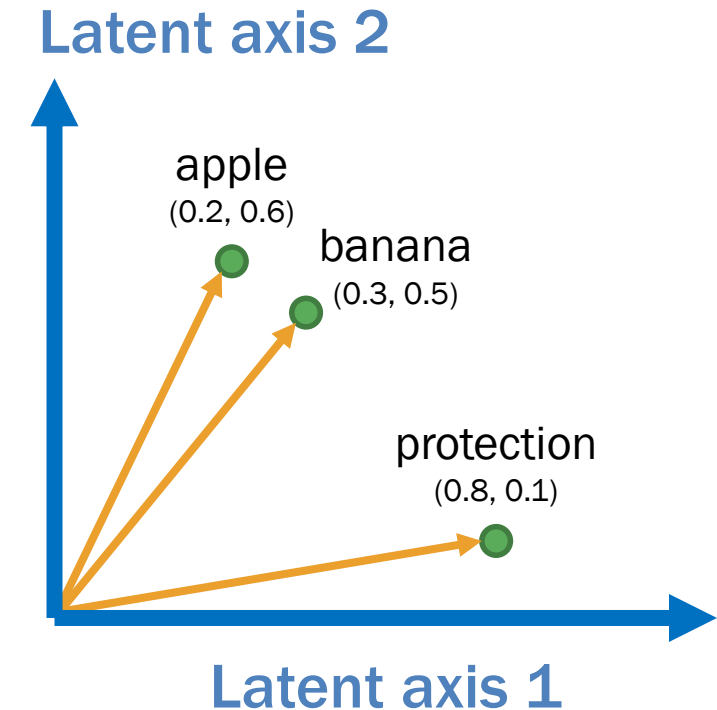
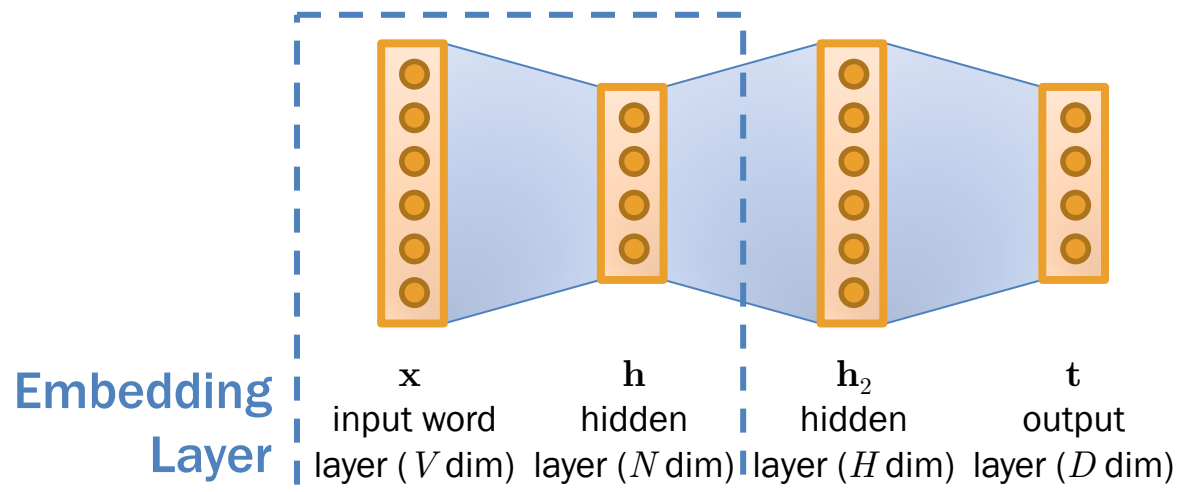
- **Encoding vector $\mathbf{v}_I(k)$** encodes the input word x_k to a class in the hidden layer classifier
- **Decoding vector $\mathbf{v}_O(j)$** decodes the class in the hidden layer to a word y_j in the output layer

$$\mathbf{v}_I(k) = \begin{bmatrix} a_{k,1} & a_{k,2} & a_{k,3} & \dots & a_{k,N} \end{bmatrix}^\top$$

$$\mathbf{v}_O(j) = \begin{bmatrix} b_{j,1} & b_{j,2} & b_{j,3} & \dots & b_{j,N} \end{bmatrix}^\top$$

$\mathbf{v}_I(k)$ a.k.a. Embedding Layer

- Encoding vector $\mathbf{v}_I(k)$ represents the input word w_k in the latent vector space
- Output layer $\mathbf{v}_O(j)$ can be replaced by other networks w.r.t. your task
 - E.g. POS tagging with Word2Vec



Popular Word2Vec libraries (e.g. gensim) return $\mathbf{v}_I(\text{word})$ as the word representation

2. Word Embedding Models (cont'd)

- Neural network for word co-occurrence

- Encoding: $\mathbf{h}_{(k)} = \mathbf{v}_I(k)$

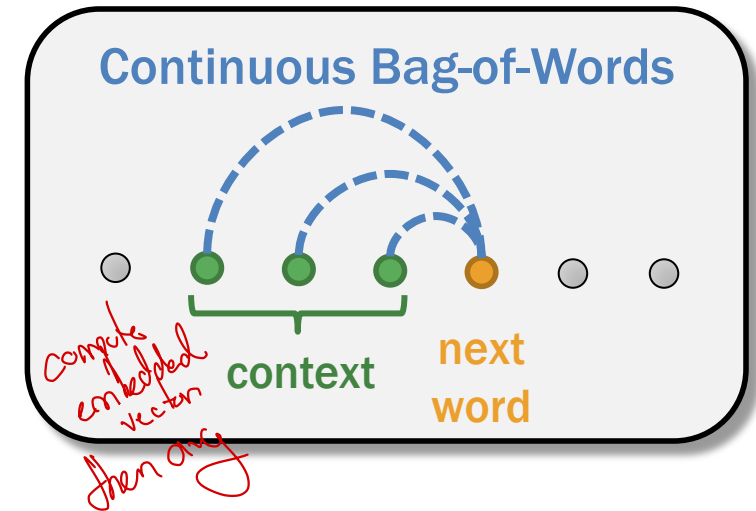
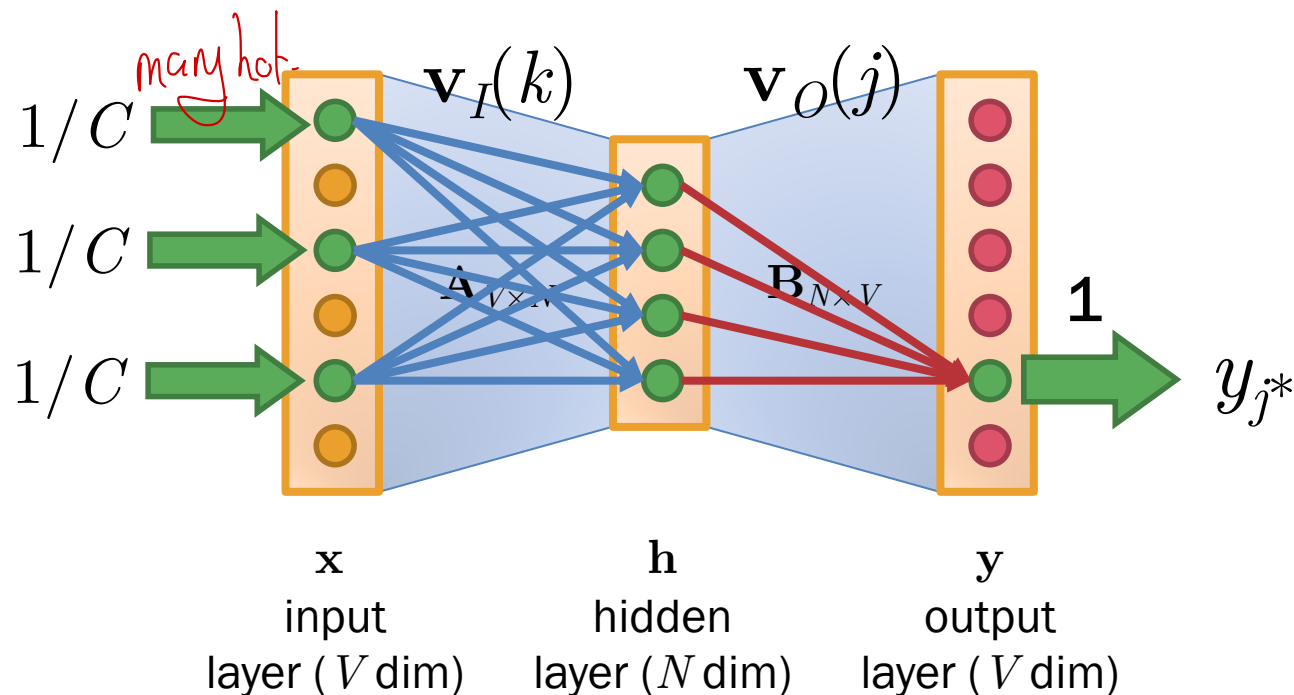
- Decoding: $\mathbf{u}_{(j)} = \mathbf{v}_O^\top(j) \mathbf{v}_I(k)$

- Posterior:
$$p(w_O = y_{j^*} | w_I) = \frac{\exp u_{j^*}}{\sum_{j=1}^V \exp u_j} \quad (\text{softmax})$$

- Word embedding:
$$\mathbf{v}_I(k) = \begin{bmatrix} a_{k,1} & a_{k,2} & a_{k,3} & \dots & a_{k,N} \end{bmatrix}^\top$$
$$\mathbf{v}_O(j) = \begin{bmatrix} b_{j,1} & b_{j,2} & b_{j,3} & \dots & b_{j,N} \end{bmatrix}^\top$$

Continuous Bag-of-Words (Harris, 1954)

- We simultaneously input several context words (C) and predict one output word

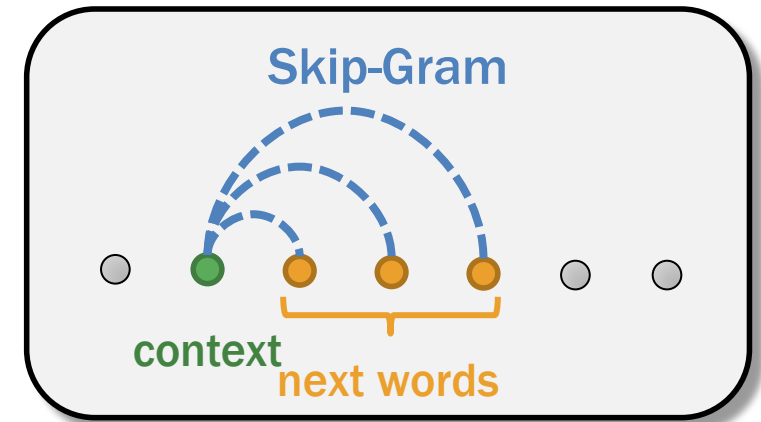
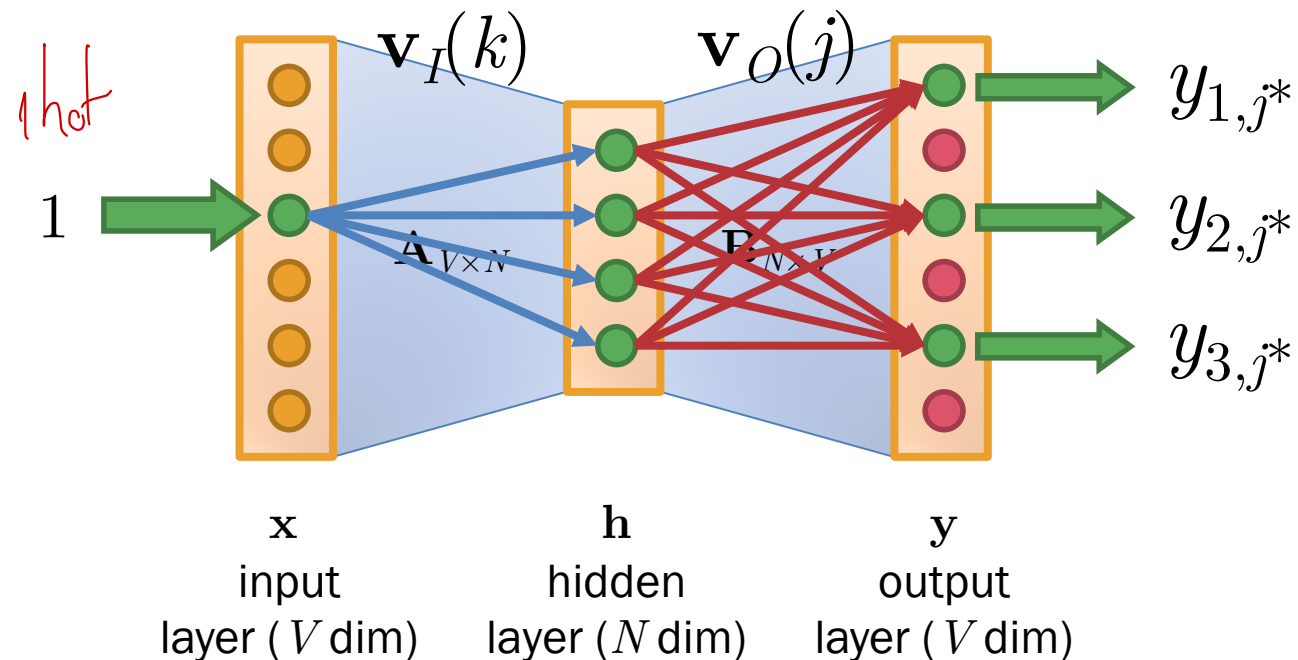


- We simply average the encoding vectors

$$\mathbf{h}_{\text{ctx}} = \frac{1}{C} \sum_{q=1}^C \mathbf{v}_I(q)$$

Skip Gram (Mikolov+, 2013)

- Several output words (C) are predicted independently from one context word



- Each output word is generated independently

$$\mathbf{u}_{c,(j)} = \mathbf{v}_O^\top(j_c) \mathbf{v}_I(k)$$

make them independently

3. Training with Backpropagation

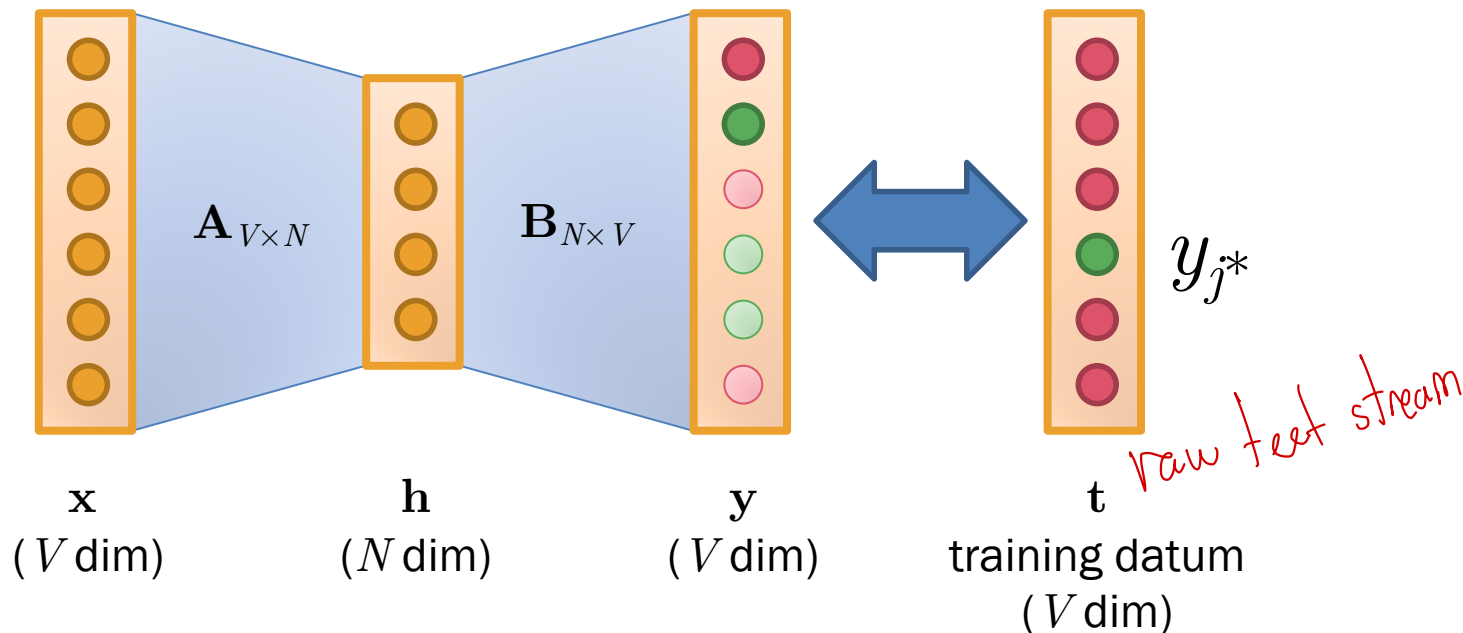
3. Training with Backpropagation

- Minimizing errors of the prediction score w.r.t. the training set
- Therefore, maximizing $p(w_O = y_{j^*} | w_I = x_k)$

[Hello, my, name, is, Boon]

$$p(my) = (v_o(my)^T v_i(hello))$$

maximize the probability, NLL (-log)

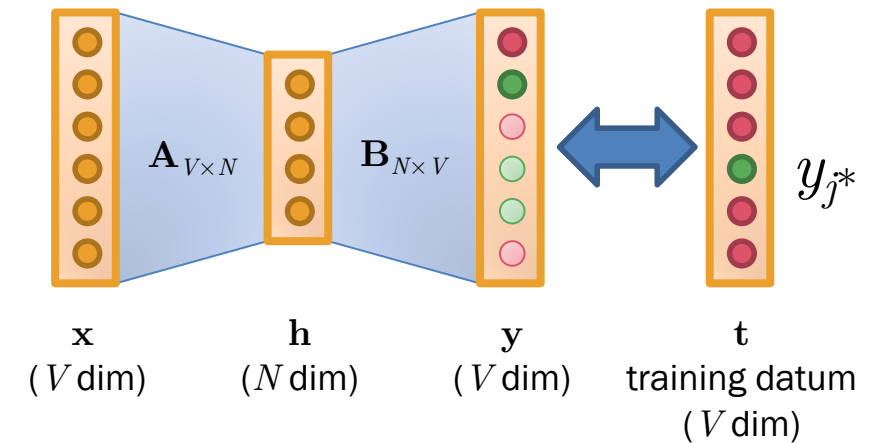


3. Training with Backpropagation (cont'd)

- To maximize $p(w_O = y_{j^*} | w_I = x_k)$, we want to minimize the prediction error E

$$\begin{aligned} E &= -\log p(w_{j^*} | w_I) \\ &= -\log \frac{\exp u_{j^*}}{\sum_j \exp u_j} \\ &= (\log \sum_j \exp u_j) - u_{j^*} \end{aligned}$$

- Note that u_{j^*} is the prediction score of the correct output



$$E = (\log \sum_j \exp u_j) - u_{j^*}$$

$$\begin{aligned} \log(AB) &= \log A + \log B \\ \log\left(\frac{A}{B}\right) &= \log A - \log B \\ \log(\exp A) &= \exp(\log A) = A \end{aligned}$$

3. Training with Backpropagation (cont'd)

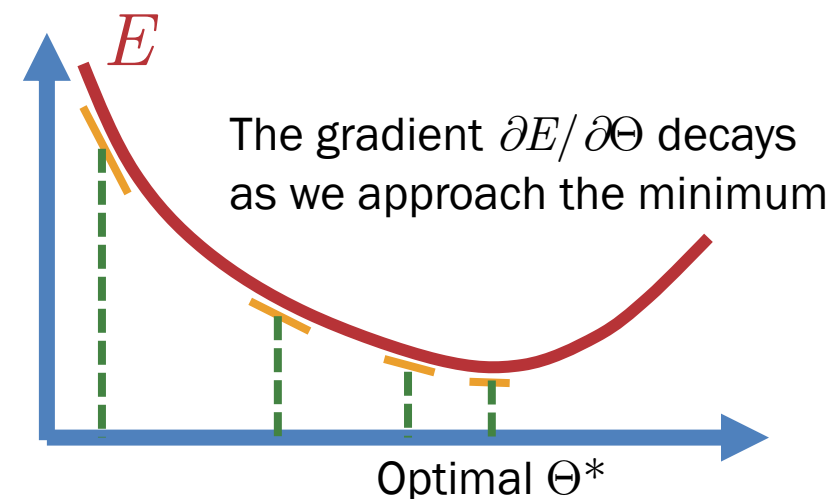
- **Stochastic Gradient Descent (SGD)**

- We want to approximate the model parameters Θ such that the prediction error becomes minimized at an optimal Θ^*

- **Algorithm:**

1. Set the initial value of the model parameters Θ
2. Update the model parameters Θ w.r.t. the gradient $\partial E / \partial \Theta$ at the current point
3. Repeat step 2 until the model parameters Θ converge (the gradient is close to zero)

Many more optimization methods, e.g. regularizations (L_1 , L_2 , VB), γ -momentum, Nesterov, AdaGrad, AdaDelta, RMSprop, ADAM, etc.



Update Equation

$$\Theta^{\text{new}} = \Theta^{\text{old}} - \eta \frac{\partial E}{\partial \Theta}$$

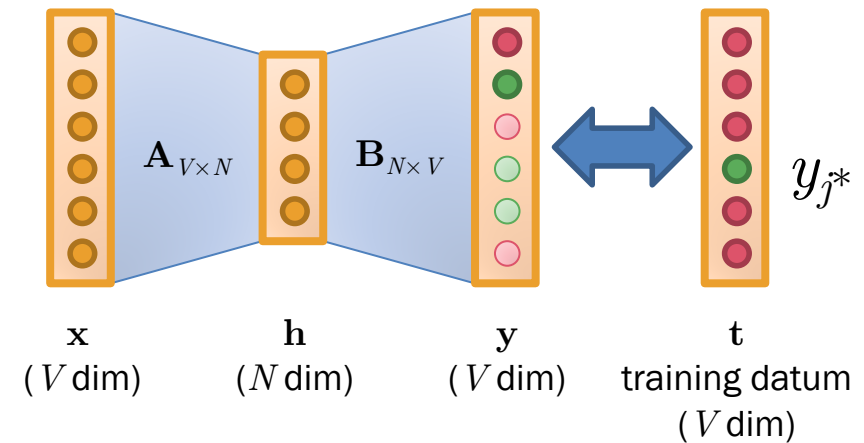
where η is the learning rate (usu. a small number such as 10^{-4})

3. Training with Backpropagation (cont'd)

- The derivative of E on b_{ij} is:

$$\begin{aligned}
 \frac{\partial E}{\partial b_{ij}} &= \frac{\partial E}{\partial u_j} \cdot \frac{\partial u_j}{\partial b_{ij}} \\
 &= e_j \cdot \frac{\partial}{\partial b_{ij}} \left(\sum_i b_{ij} h_i \right) \\
 &= e_j h_i
 \end{aligned}$$

$w_j = \sum_i b_{ij} h_i$



$$E = (\log \sum_j \exp u_j) - u_j^*$$

$$\frac{\partial E}{\partial u_j} = p(w_{j^*} | w_I) - t_j \triangleq e_j$$

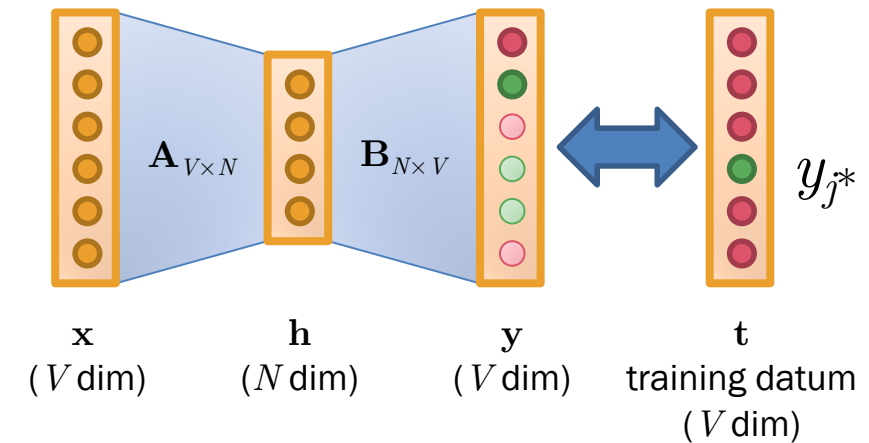
$$\frac{\partial E}{\partial b_{ij}} = e_j h_i$$

Term	Meaning
u_j	Raw logit for word j
$\hat{y}_j$	Softmax probability
$e_j = \hat{y}_j - t_j$	Error for word j
h_i	Activation of hidden neuron i
$\frac{\partial E}{\partial b_{ij}}$	Gradient of loss w.r.t. decoder weight

3. Training with Backpropagation (cont'd)

- The derivative of E on h_i is:

$$\begin{aligned}
 \frac{\partial E}{\partial h_i} &= \sum_j \left(\frac{\partial E}{\partial u_j} \cdot \frac{\partial u_j}{\partial h_i} \right) \\
 &= \sum_j \left(e_j \cdot \frac{\partial}{\partial h_i} \left(\sum_i b_{ij} h_i \right) \right) \\
 &= \sum_j e_j b_{ij} \quad \triangleq \text{EH}_i \\
 &\quad \text{(The error of the } i\text{-th hidden unit)}
 \end{aligned}$$



$$E = \left(\log \sum_j \exp u_j \right) - u_{j^*}$$

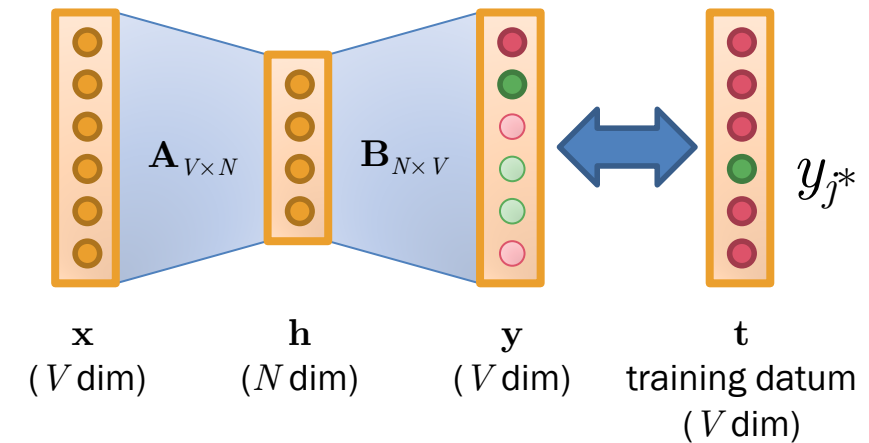
$$\frac{\partial E}{\partial u_j} = p(w_{j^*} | w_I) - t_j \triangleq e_j$$

$$\frac{\partial E}{\partial b_{ij}} = e_j h_i \quad \frac{\partial E}{\partial h_i} = \sum_j e_j b_{ij} \triangleq \text{EH}_i$$

3. Training with Backpropagation (cont'd)

- The derivative of E on a_{ki} is:

$$\begin{aligned}\frac{\partial E}{\partial a_{ki}} &= \frac{\partial E}{\partial h_i} \cdot \frac{\partial h_i}{\partial a_{ki}} \\ &= \text{EH}_i \cdot \frac{\partial}{\partial a_{ki}} \left(\sum_k a_{ki} x_k \right) \\ &= \text{EH}_i \cdot x_k\end{aligned}$$



$$E = \left(\log \sum_j \exp u_j \right) - u_{j^*}$$

$$\frac{\partial E}{\partial u_j} = p(w_{j^*} | w_I) - t_j \triangleq e_j$$

$$\frac{\partial E}{\partial b_{ij}} = e_j h_i \quad \frac{\partial E}{\partial h_i} = \sum_j e_j b_{ij} \triangleq \text{EH}_i$$

$$\frac{\partial E}{\partial a_{ki}} = \text{EH}_i \cdot x_k$$

3. Training with Backpropagation (cont'd)

- Updating $\mathbf{v}_I(k)$ and $\mathbf{v}_O(j)$ with SGD
 - For each output word w_j we compute

update vector
for input

$$\mathbf{v}_I^{\text{new}}(k) = \mathbf{v}_I^{\text{old}}(k) - \eta \frac{\partial E}{\partial \mathbf{A}_{(k, \cdot)}}$$

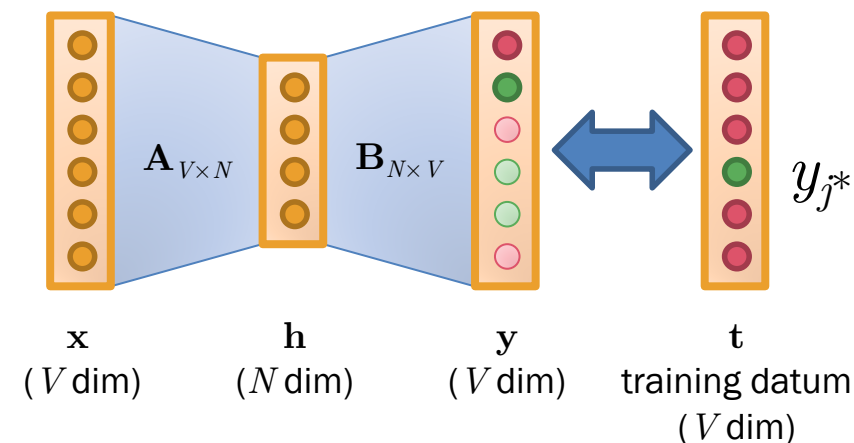
$$= \mathbf{v}_I^{\text{old}}(k) - \eta \cdot \mathbf{E}\mathbf{H} \cdot x_k$$

$$\mathbf{v}_O^{\text{new}}(j) = \mathbf{v}_O^{\text{old}}(j) - \eta \frac{\partial E}{\partial \mathbf{B}_{(j, \cdot)}}$$

output.

$$= \mathbf{v}_O^{\text{old}}(j) - \eta \cdot e_j \cdot \mathbf{h}$$

- where $\mathbf{h} = \mathbf{v}_I(k)$ and $\mathbf{E}\mathbf{H} = \mathbf{B}\mathbf{e}$
- N.B. The gradients contain errors e_j



$$E = (\log \sum_j \exp u_j) - u_{j^*}$$

$$\frac{\partial E}{\partial u_j} = p(w_{j^*} | w_I) - t_j \triangleq e_j$$

$$\frac{\partial E}{\partial b_{ij}} = e_j h_i \quad \frac{\partial E}{\partial h_i} = \sum_j e_j b_{ij} \triangleq \mathbf{E}\mathbf{H}_i$$

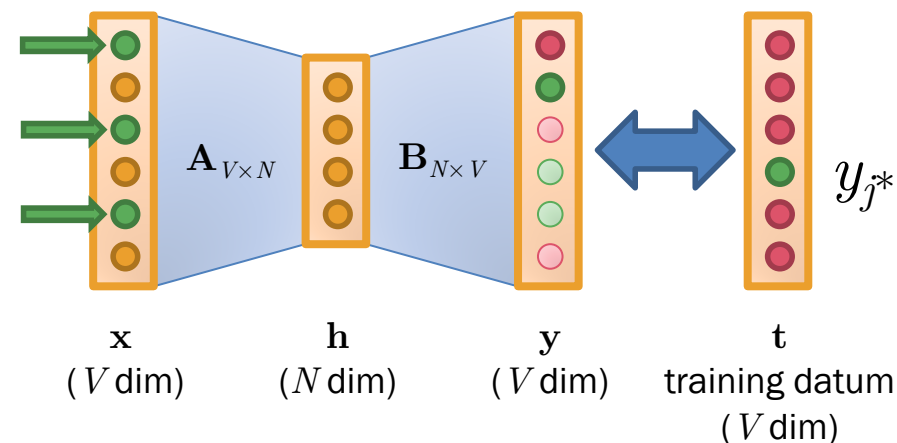
$$\frac{\partial E}{\partial a_{ki}} = \mathbf{E}\mathbf{H}_i \cdot x_k$$

Continuous Bag-Of-Words (Harris, 1954)

- There are C input words
- By SGD, we obtain the following updates

$$\begin{aligned}
 \mathbf{v}_I^{\text{new}}(k) &= \mathbf{v}_I^{\text{old}}(k) - \eta \frac{\partial E}{\partial \mathbf{A}_{(k, \cdot)}} \\
 &= \mathbf{v}_I^{\text{old}}(k) - \eta \cdot \mathbf{E} \mathbf{H} \cdot x_k \quad \text{Always } 1/C \\
 \mathbf{v}_O^{\text{new}}(j) &= \mathbf{v}_O^{\text{old}}(j) - \eta \frac{\partial E}{\partial \mathbf{B}_{(j, \cdot)}} \\
 &= \mathbf{v}_O^{\text{old}}(j) - \eta \cdot e_j \cdot \mathbf{h}
 \end{aligned}$$

- where $\mathbf{h} = \mathbf{v}_I(k)$ and $\mathbf{E} \mathbf{H} = \mathbf{B} \mathbf{e}$
- N.B. The gradients contain errors e_j



$$E = (\log \sum_j \exp u_j) - u_{j^*}$$

$$\frac{\partial E}{\partial u_j} = p(w_{j^*} | w_I) - t_j \triangleq e_j$$

$$\frac{\partial E}{\partial b_{ij}} = e_j h_i \quad \frac{\partial E}{\partial h_i} = \sum_j e_j b_{ij} \triangleq \mathbf{E} \mathbf{H}_i$$

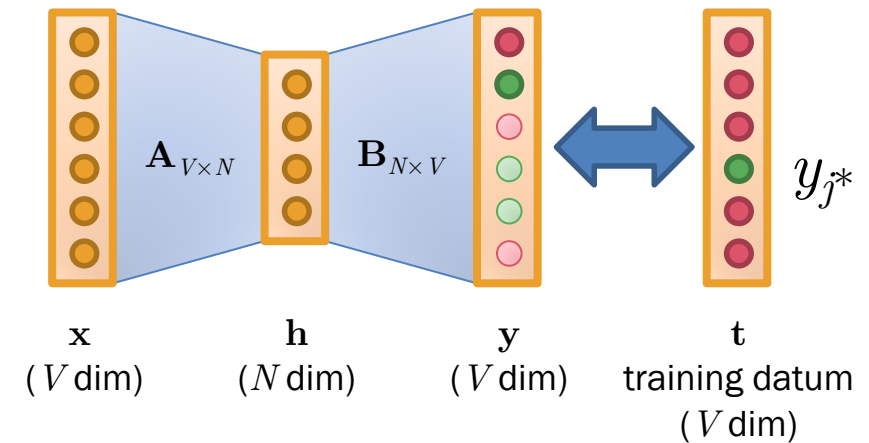
$$\frac{\partial E}{\partial a_{ki}} = \mathbf{E} \mathbf{H}_i \cdot x_k$$

Skip Gram (Mikolov+, 2013)

- Each output word is independent
 - By SGD, we obtain the following updates

$$\begin{aligned}
 \mathbf{v}_I^{\text{new}}(k) &= \mathbf{v}_I^{\text{old}}(k) - \eta \frac{\partial E}{\partial \mathbf{A}_{(k, \cdot)}} \\
 &= \mathbf{v}_I^{\text{old}}(k) - \eta \cdot \mathbf{E} \mathbf{H} \cdot x_k \quad \text{Always 1} \\
 \mathbf{v}_O^{\text{new}}(j) &= \mathbf{v}_O^{\text{old}}(j) - \eta \frac{\partial E}{\partial \mathbf{B}_{(j, \cdot)}} \\
 &= \mathbf{v}_O^{\text{old}}(j) - \eta \cdot e_j \cdot \mathbf{h}
 \end{aligned}$$

- where $\mathbf{h} = \mathbf{v}_I(k)$ and $\mathbf{E} \mathbf{H} = \mathbf{B} \mathbf{e}$
- N.B. The gradients contain errors e_j



$$E = (\log \sum_j \exp u_j) - u_{j^*}$$

$$\frac{\partial E}{\partial u_j} = p(w_{j^*} | w_I) - t_j \triangleq e_j$$

$$\frac{\partial E}{\partial b_{ij}} = e_j h_i \quad \frac{\partial E}{\partial h_i} = \sum_j e_j b_{ij} \triangleq \mathbf{E} \mathbf{H}_i$$

$$\frac{\partial E}{\partial a_{ki}} = \mathbf{E} \mathbf{H}_i \cdot x_k$$

4. Performance Optimization

4. Performance Optimization (Mikolov+, 2013)

- Computing the vectors for natural language (700k+ vocab) is challenging
 - Looping over $v_o(j)$ is laborious!
 - [P: Space] Replace the one-hot output with a more compact representation: **branching factorization**
 - [P: Speed] Ignore some insignificant output words that rarely show up in the dataset: **negative sampling**
 - [P: Coverage] Learning stop words is meaningless: **frequent word subsampling**



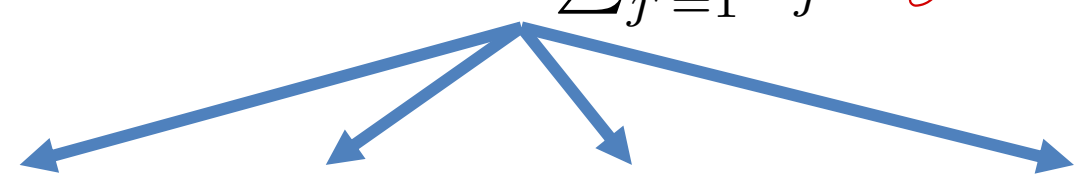
4.1 Hierarchical Softmax (Mnih+, 2009)

- We decompose a softmax posterior σ over a large set of instances into branching softmax distributions

Softmax over Softmax

$$p(y_j | w_I) = \frac{\exp u_j}{\sum_{j'=1}^V u_{j'}}$$

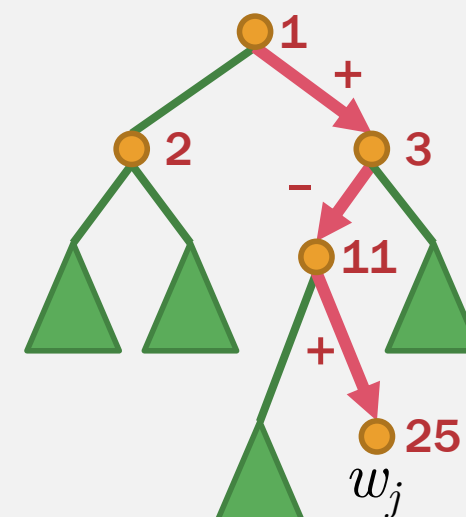
Divide vector into parts



$$\sigma(\mathbf{v}_O(q_{j,1})\mathbf{h}) \quad \sigma(\mathbf{v}_O(q_{j,2})\mathbf{h}) \quad \sigma(\mathbf{v}_O(q_{j,3})\mathbf{h}) \quad \dots \quad \sigma(\mathbf{v}_O(q_{j,L})\mathbf{h})$$

- where each $\mathbf{v}_O(q_{j,n})$ is the decoder vector for the right branching of node n

Branching Factorization



softmax

$$\sigma(w_j) = \sigma(+\mathbf{v}_O(n_1)\mathbf{h}) \times \sigma(-\mathbf{v}_O(n_3)\mathbf{h}) \times \sigma(+\mathbf{v}_O(n_{11})\mathbf{h})$$

softmax

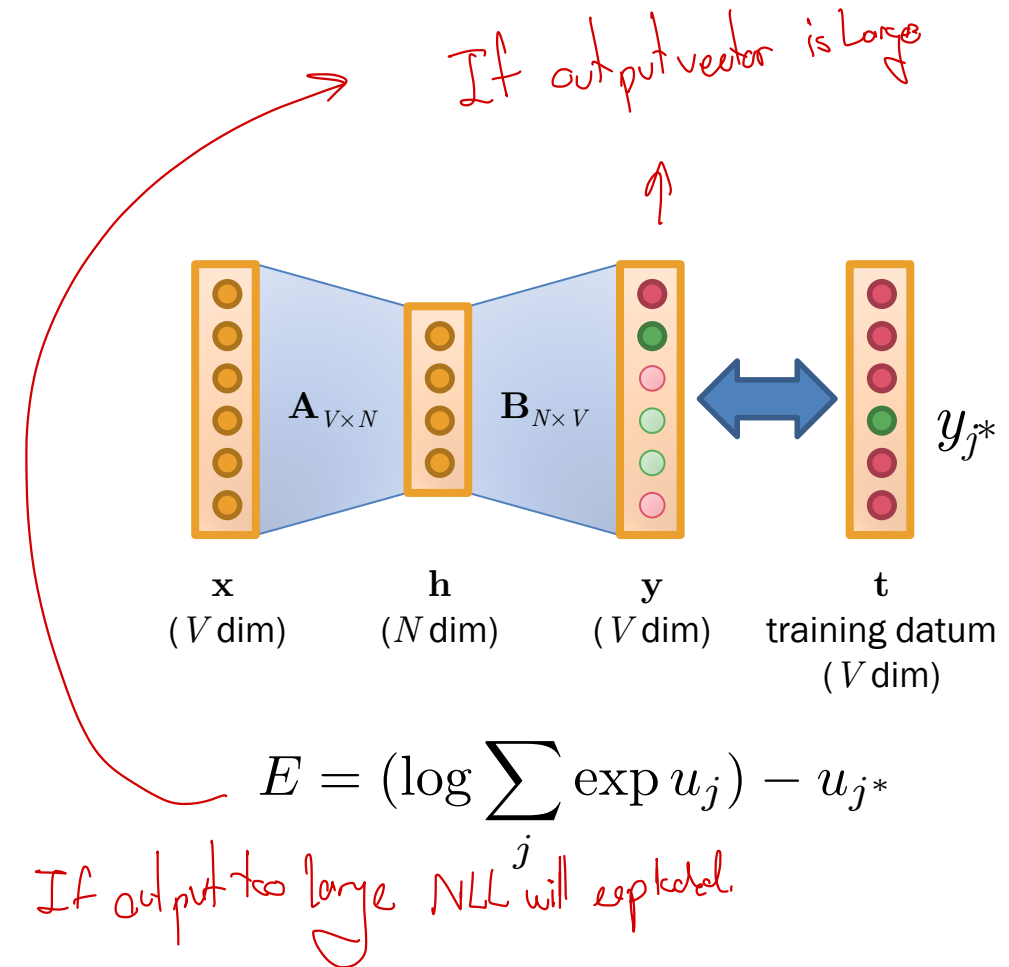
softmax

Plugging in to Word Embedding Models

- We replace output word w_j with $q_{j,n}$
- Update equations:

$$\begin{aligned}\mathbf{v}_I^{\text{new}}(k) &= \mathbf{v}_I^{\text{old}}(k) - \eta \frac{\partial E}{\partial \mathbf{A}_{(k, \cdot)}} \\ &= \mathbf{v}_I^{\text{old}}(k) - \eta \cdot \mathbf{E} \mathbf{H} \cdot x_k \\ \mathbf{v}_O^{\text{new}}(q_{j,n}) &= \mathbf{v}_O^{\text{old}}(q_{j,n}) - \eta \frac{\partial E}{\partial \mathbf{B}_{(q_{j,n}, \cdot)}} \\ &= \mathbf{v}_O^{\text{old}}(q_{j,n}) - \eta \cdot e_{q_{j,n}} \cdot \mathbf{h}\end{aligned}$$

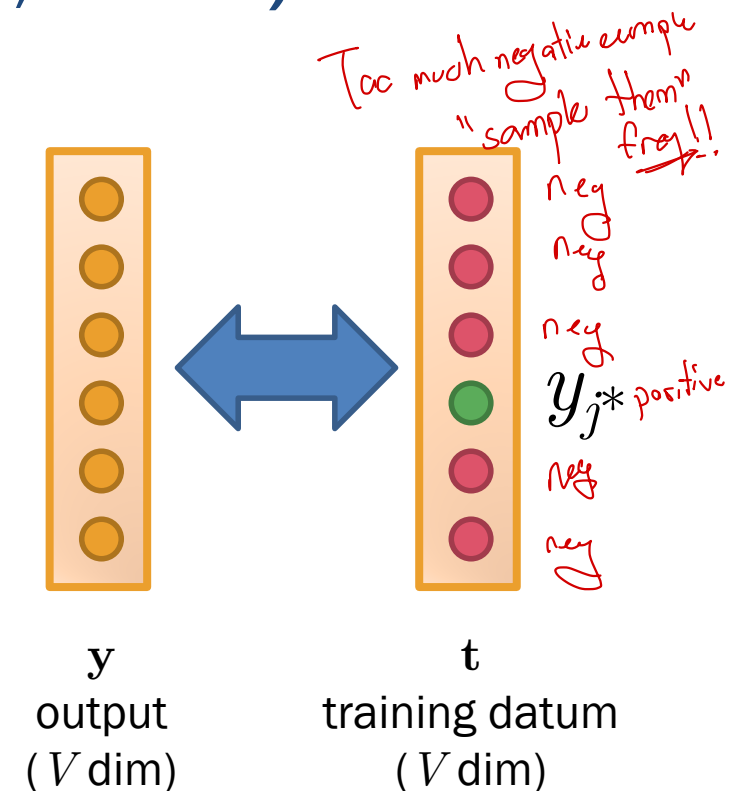
- where $\mathbf{h} = \mathbf{v}_I(k)$ and $\mathbf{E} \mathbf{H} = \mathbf{B} \mathbf{e}$



Training by Positive & Negative example.

4.2 Negative Sampling (Mikolov+, 2013)

- We assume that the dataset is noisy, containing
 - **Positive examples:** correct output words
 - **Negative examples:** incorrect output words
- How to reduce the computational cost
 - The **positive examples** should be kept
 - Not all **negative examples** need to be considered; i.e. we can ignore the insignificant ones that rarely show up



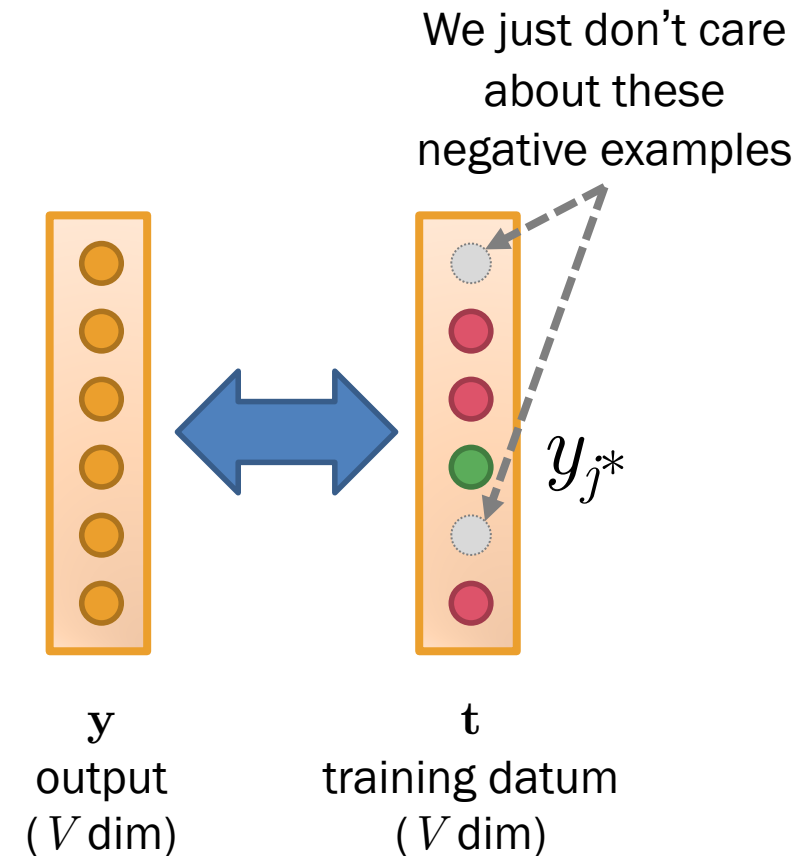
Noise Contrastive Estimation (Gutmann+, 2012)

Plugging in to Word Embedding Models

- The prediction error contains noise:

$$E = -\log \frac{u_{j^*}}{\sum_j u_j} - \sum_{w_q \in W_{\text{neg}}} \log \frac{u_q}{\sum_j u_j}$$

- where each $w_q \in W_{\text{neg}}$ is sampled from a distribution, e.g. $w_q \sim p_{\text{unigram}}(w_q)^{3/4}$
f word powered by (3/4)
- Algorithm:**
 - Update the parameters for **the positive** w_o
 - Sample only some **negative examples** w_q and update the parameters only for them

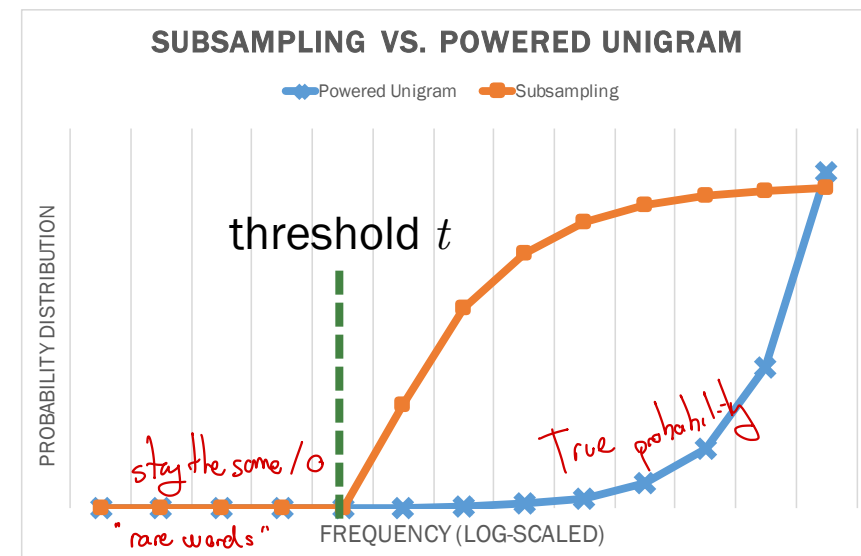


4.3 Freq Word Subsampling (Mikolov+, 2013)

- Learning stop words is meaningless but listing all of them is yet laborious
- $p_{\text{unigram}}(w_j)^{3/4}$ alone is inadequate due to extreme data imbalance in big data
- In negative sampling, we sample negative examples from this subsampling distribution

$$p_{\text{neg}}(w_j) \propto \max \left(1 - \sqrt{\frac{t}{p_{\text{unigram}}(w_j)}}, 0 \right)$$

- where t is a chosen threshold (usu. 10^{-5})



Subsampling: how it works

- It increases the chance of rare words above the threshold to be chosen
- It eliminates extremely rare words by truncating the unigram distribution
- It is also useful for other tasks involving stop words

ADAM $\rightarrow$ based on this

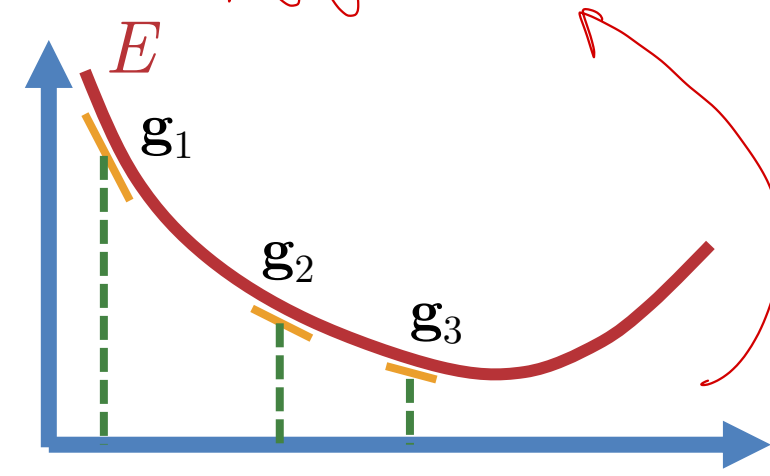
4.4 AdaGrad (Duchi+, 2011)

- [P: Speed] **Adaptive gradient algorithm** automatically controls the learning rate to avoid sparse parameters
- Sparseness of parameter Θ at τ -th iteration is the L_2 -norm of subgradients

converge faster! $G_{\Theta}^{(\tau)} = \left(\sum_{i=1}^{\tau} \mathbf{g}_i^{\top} \mathbf{g}_i \right)^{1/2}$

where each $\mathbf{g}_i = \partial E / \partial \Theta$ at i -th iteration

Look at last 3 gradient, compute $\rightarrow$ the stopping gradient



$$G_{\Theta}^{(3)} = \sqrt{\mathbf{g}_1^{\top} \mathbf{g}_1 + \mathbf{g}_2^{\top} \mathbf{g}_2 + \mathbf{g}_3^{\top} \mathbf{g}_3}$$

Look at last 3 gradient

Update Equation

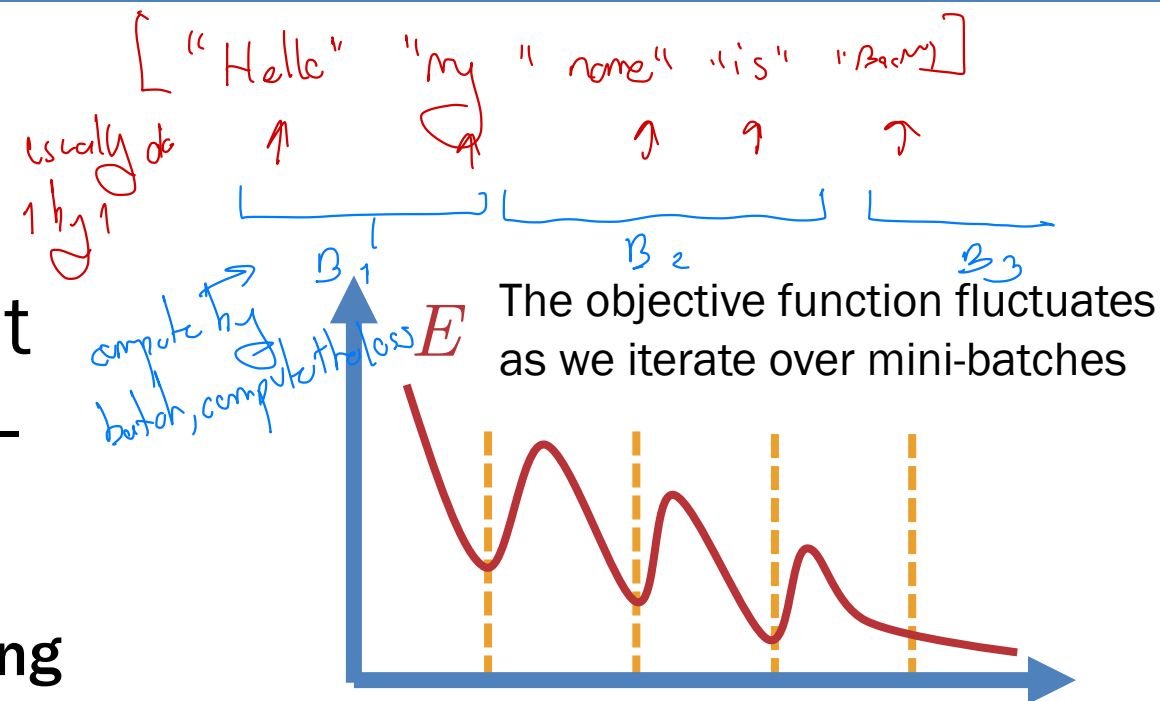
$$\Theta^{(\tau+1)} = \Theta^{(\tau)} - \frac{\eta}{G_{\Theta}^{(\tau)}} \mathbf{g}^{(\tau)}$$

where η is the learning rate (usu. a small number such as 10^{-4})

4.5 Mini-Batch Training

Batch Training.

- **[P: Speed]** We compute the gradient $\partial E / \partial \Theta$ on small datasets ('mini-batches') to boost parallelism



Traditional computation

$$f\left(\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix} + \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}\right) = \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}$$

$$f\left(\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix} + \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}\right) = \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}$$

$$f\left(\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix} + \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}\right) = \begin{bmatrix} \cdot \\ \cdot \\ \cdot \end{bmatrix}$$

Mini-batching

$$f\left(\begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix} + \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}\right) \approx \begin{bmatrix} \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$$

where $\mathbf{X} = \begin{bmatrix} \mathbf{x}_1 & \mathbf{x}_2 & \mathbf{x}_3 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$ mini-batch

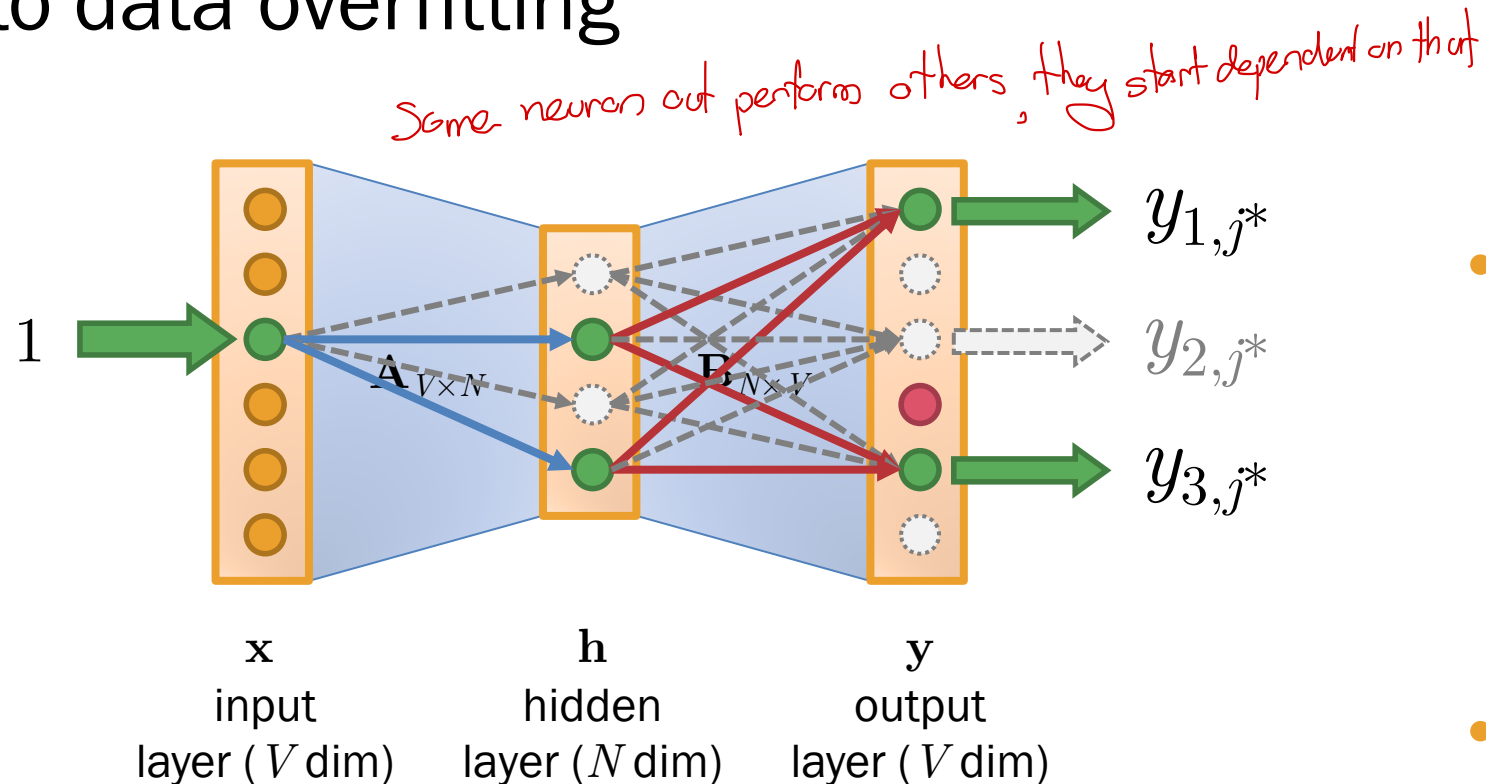
and $\mathbf{Y} = \begin{bmatrix} \mathbf{y}_1 & \mathbf{y}_2 & \mathbf{y}_3 \\ \cdot & \cdot & \cdot \\ \cdot & \cdot & \cdot \end{bmatrix}$

Algorithm

- We separate the dataset into mini-batches [size = 20 to 1,000]
- For each mini-batch, we compute the gradient of all elements in parallel using vectorization and update the model parameters Θ
- Repeat until convergence

4.6 Dropout (Hinton+, 2012)

- **[P: Coverage]** Non-linear classifiers are prone to data overfitting



- Lazy neurons doesn't learn anything*
- **Co-adaptation:** Some features mistakenly become dependent on each other
 - **Solution:** We ignore ~50% of the units of each layer in each training iteration at random *kick smart one out*
 - **Highly recommended**

5. Statistical Interpretation

5. Statistical Interpretation (Levy+, 2014)

- We will show that $\mathbf{v}_O(j)^\top \mathbf{v}_I(k)$ is statistically meaningful
 - $\#(j)$ = frequency of the j^{th} output word
 - $\#(k)$ = frequency of the k^{th} input word
 - $\#(j,k)$ = frequency of the pair (w_j, w_k)
- For each pair (w_j, w_k) in training set D

Sigmoid Function

$$\sigma(r) = \frac{1}{1 + \exp(-r)}$$

$$E = \#(j, k) \cdot \log \sigma(\mathbf{v}_O(j)^\top \mathbf{v}_I(k)) \quad \text{Positive examples}$$

$$+ \#(k) \cdot \underbrace{\frac{\#(j) |W_{\text{neg}}|}{|D|}}_{\text{Because } w_j \sim p_{\text{unigram}}(w_j)} \cdot \log \sigma(-\mathbf{v}_O(j)^\top \mathbf{v}_I(k)) \quad \text{Negative examples}$$

5. Statistical Interpretation (Levy+, 2014) (cont'd)

- To optimize the objective function E , we let $r = \mathbf{v}_O(j)^\top \mathbf{v}_I(k)$

$$E = \#(j, k) \cdot \log \sigma(r) + \frac{\#(j)\#(k)|W_{\text{neg}}|}{|D|} \cdot \log \sigma(-r)$$

- The derivative of E on r becomes

$$\frac{\partial E}{\partial r} = \#(j, k) \cdot \sigma(-r) - \frac{\#(j)\#(k)|W_{\text{neg}}|}{|D|} \cdot \sigma(r)$$

- We then set $\partial E / \partial r = 0$

5. Statistical Interpretation (Levy+, 2014) (cont'd)

- After solving it, we arrive at

$$\exp(2r) - \left(\frac{\#(k, j)|D|}{|W_{\text{neg}}|\#(k)\#(j)} - 1 \right) \exp(r) - \frac{\#(k, j)|D|}{|W_{\text{neg}}|\#(k)\#(j)} = 0$$

- Let $s = \exp(r)$, we have

$$s = \exp(\mathbf{v}_O(j)^\top \mathbf{v}_I(k)) = \frac{\#(k, j)|D|}{|W_{\text{neg}}|\#(k)\#(j)}$$

- We finally obtain that

Pointwise Mutual Information (PMI) (Church+, 1990)

$$\mathbf{v}_O(j)^\top \mathbf{v}_I(k) = \boxed{\log \frac{\#(k, j)}{\#(k)\#(j)}} + \log |D| - \log |W_{\text{neg}}|$$

5. Statistical Interpretation (Levy+, 2014) (cont'd)

- What does it mean?

Pointwise Mutual Information (PMI) (Church+, 1990)

$$\mathbf{v}_O(j)^\top \mathbf{v}_I(k) = \boxed{\log \frac{\#(k, j)}{\#(k) \#(j)}} + \underbrace{\log |D| - \log |W_{\text{neg}}|}_{\text{constant}}$$

- PMI is a measure for statistical association between two instances in an event
- So, $\mathbf{v}_O(j)^\top \mathbf{v}_I(k)$ determines how similar the input word $w_I = x_k$ and the predicted word $w_O = x_j$ are w.r.t. their contexts

6. Experiments as in Literature

6. Experiments as in Literature

- **Experiment 1** (Mikolov+, 2013)
 - Objective: Finding the best setting
 - Task: **Analogical reasoning** (Mikolov+, 2013)
 - E.g. **Germany** :: **Berlin** vs. **France** :: ??
 - Two test sets: Syntax and Semantics
 - Training set: Google's Gigawords
 - Words with <5 times occurrence are discarded
 - Containing 692K distinct words
 - Metric: accuracy (= correct / total)

Google's Best Practice

Hidden layer	300 units 1,000 units
Context size	5 words
Binary tree	Huffman code
Subsampling threshold	10^{-5}
Learning rate	10^{-4}
Optimizer	AdaGrad

Task's file: <http://code.google.com/p/word2vec/source/browse/trunk/questions-words.txt>

6. Experiments as in Literature (cont'd)

- **Experiment 1** (Mikolov+, 2013)

Methods	Training Time	Syntax [%]	Semantics [%]	Total Accuracy [%]
NEG (5 neg examples)	<u>38 mins</u>	<u>63</u>	54	59
NEG (15 neg examples)	97 mins	<u>63</u>	<u>58</u>	<u>61</u>
HierSoftmax (Huffman)	41 mins	53	40	47
NCE (logistic regression)	38 mins	60	45	53
NEG (5 neg examples) + Subsampling	<u>14 mins</u>	<u>61</u>	58	60
NEG (15 examples) + Subsampling	36 mins	<u>61</u>	<u>61</u>	<u>61</u>
HierSoftmax (Huffman) + Subsampling	21 mins	52	59	55

6. Experiments as in Literature (cont'd)

- **Experiment 1** (Mikolov+, 2013)
 - Negative sampling
 - Is faster to train than hierarchical softmax
 - Yields better accuracy
 - Training time $\propto$ size of neg examples (linearly)
 - Frequent word subsampling
 - Boosts the training speed
 - Deteriorates syntax coverage because frequent function words are less chosen
 - Boosts semantic coverage because the meaning is expressed by rare content words

6. Experiments as in Literature (cont'd)

- **Phrase learning** (Mikolov+, 2013)

- Bracketing score

$$\text{score}(w_i, w_j) = \frac{\#(w_i, w_j) - \delta}{\#(w_i) \times \#(w_j)}$$

- where δ is a discounting coefficient

4-gram: **The_Thai_Airways_International**



trigram: **Thai_Airways_International**



bigram: **Thai_Airways**



The Thai Airways International has reopened the New York route

???

Phrase Chunking Algo

1. For each input string, merge any two tokens w_i and w_{i+1} if $\text{score}(w_i, w_{i+1}) > \text{threshold } \theta$
2. Decrease the threshold θ
3. Repeat steps 1 and 2 for 2-4 passes (resulting in up to 5-grams)

6. Experiments as in Literature (cont'd)

- **Experiment 2** (Mikolov+, 2013)
 - Objective: Phrase learning
 - Task: Analogical reasoning
 - E.g. Belgium :: Brussels Airlines vs. United Kingdom :: ??
 - Training set: Google's Gigawords with the same settings as Experiment 1
 - Metric: accuracy (= correct / total)
 - **Observation:** The phrase distribution is much sparser than the word distribution

Google's Best Practice

Hidden layer	300 units 1,000 units
Context size	5 words
Binary tree	Huffman code
Subsampling threshold	10^{-5}
Learning rate	10^{-4}
Optimizer	AdaGrad

Task's file: <http://code.google.com/p/word2vec/source/browse/trunk/questions-phrases.txt>

6. Experiments as in Literature (cont'd)

- **Experiment 2** (Mikolov+, 2013)

context size = entire sentence

Methods	Dim	Total Accuracy [%]	Dim	Accuracy [%] (6 gigawords)	Accuracy [%] (33 gigawords)
NEG (5 neg examples)	300	24			
NEG (15 neg examples)	300	<u>27</u>			
HierSoftmax (Huffman)	300	19			
NEG (5 neg examples) + Subsampling	300	27			
NEG (15 examples) + Subsampling	300	42			
HierSoftmax (Huffman) + Subsampling	300	<u>47</u>	1,000	<u>66</u>	<u>72</u>

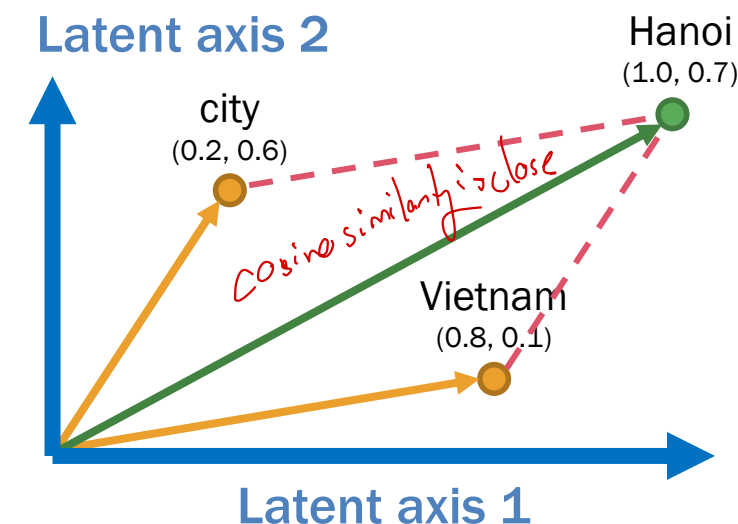
Mikolov+ (2013) chose the best setting and studied the performance upper bound

6. Experiments as in Literature (cont'd)

- **Experiment 2** (Mikolov+, 2013)
 - Most phrases are very rare and pile up at the tail of the Zipf's distribution
 - Without frequent word subsampling, phrases are much less likely to become negative examples
 - With subsampling, rare phrases become more likely to become negative examples
 - Branching softmax copes with rare phrases more efficiently than negative sampling does
 - More data improves the chance of phrases being chosen as negative examples

6. Experiments as in Literature (cont'd)

- **Additive compositionality** (Mikolov+, 2013)
 - $\mathbf{v}_I(k)$ represents the context in which the input word w_k appears
 - Mikolov+ (2013) found that $\mathbf{v}_I(\text{phrase})$ equals to the sum of $\mathbf{v}_I(\text{each word})$
 - Phrase appears in the same context as its words

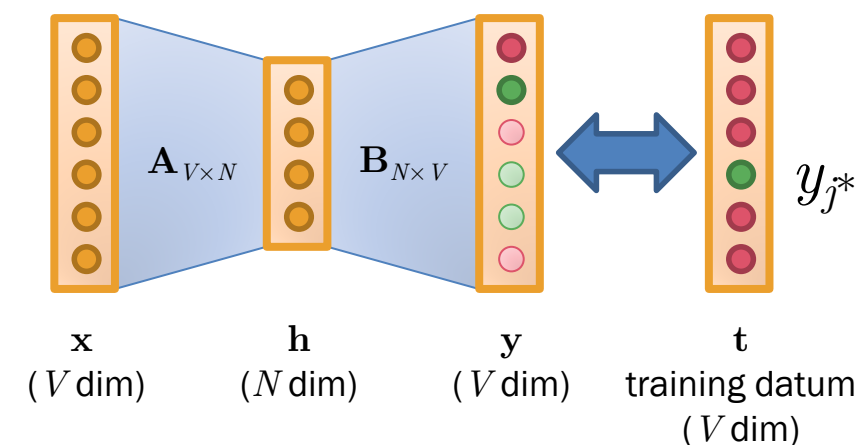
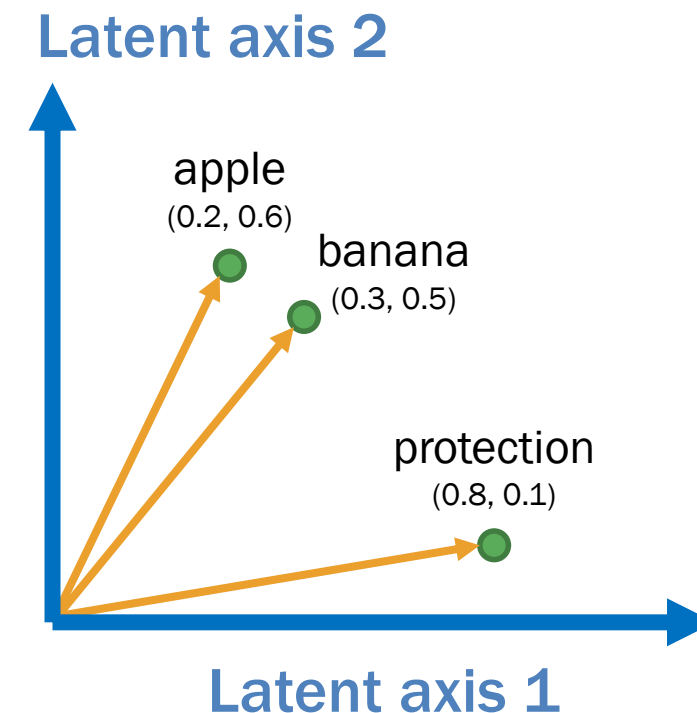


Czech + currency	Vietnam + capital	German + airlines	Russian + river	French + actress
koruna	Hanoi	airline Lufthansa	Moscow	Juliette Binoche
Czech crown	Ho Chi Minh City	carrier Lufthansa	Volga River	Vanessa Paradis
Polish złoty	Viet Nam	flag carrier Lufthansa	upriver	Charlotte Gainsbourg
CTK	Vietnamese	Lufthansa	Russia	Cecile De

Conclusion

Conclusion: Word2Vec

- Word embedding is a vector representation of words in the vector space
- Word similarity is based on massive context distribution, implying word semantics
- We can use word embedding as an input for any neural networks of interest
- Additive compositionality makes word embedding very useful for any NLP tasks
- Download: <http://code.google.com/p/word2vec>



Thank You

prachya@siit.tu.ac.th

kaamanita@gmail.com